



**COMSATS University Islamabad,
Sahiwal Campus, Pakistan**

Quran Memorization Management System

**A project presented to
COMSATS Institute of Information Technology, Sahiwal
Campus**

By

Amina Zafar	CIIT/FA22-BSE-100/SWL
Ayesha Saddiqa	CIIT/FA22-BSE-101/SWL
Dua Zafar	CIIT/FA22-BSE-128/SWL

Supervisor

Dr. Muhammad Shoaib

**In partial fulfillment
of the requirement for the degree of
Bachelor of Science in Software Engineering (2022-2026)**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Amina Zafar

Ayesha Saddiqa

Dua Zafar

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(SE) “**Quran Memorization Management System**” was developed by **Amina Zafar (CII/FA22-BSE-100)**, **Ayesha Saddiqa (CIIT/FA22-BSE-101)**, and **Dua Zafar (CIIT/FA22-BSE-128)** under the supervision of “**Dr. Muhammad Shoaib**” and co supervisor “**Mr.Arslan Sarwar**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Dr. Muhammad Shoaib

Lecturer

Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr M Munawar Iqbal

Associate Professor

Department of Computer Science
UET Taxila

Head of Department

Department of Computer Science

Dr. Zafar Iqbal Roy

Assistant Professor

Executive Summary

In today's digital age, traditional Madrasas face significant challenges in managing Quran memorization (Hifz) programs efficiently. Teachers struggle to individually assess 20 to 30 students during SAMAA (listening) sessions, leading to unnoticed recitation mistakes and inconsistent progress tracking. Students lack independent practice tools and systematic feedback mechanisms. Administrators have no data driven visibility into teacher effectiveness or overall institutional performance. Most existing solutions are either cloud dependent, lack AI capabilities, or fail to provide comprehensive Madrasa management features.

To address these critical problems, Quran Memorization Management System (QMMS) has been proposed as an all in one intelligent platform for Madrasa management and Quran memorization tracking. The purpose is to combine multiple Madrasa management features into a single, user friendly application that helps administrators, teachers, and Hifz students manage, track and improve Quran memorization outcomes. The system integrates student enrollment, teacher assignment, three shift attendance management, task assignment, AI powered recitation mistake detection, performance analytics, and financial accounting within one application.

QMMS allows administrators to manage student and teacher accounts, assign teachers to students, and monitor overall Madrasa performance. Teachers can mark attendance across three shifts, assign specific Surah and Ayah ranges as memorization tasks, and review AI generated mistake reports for each student. The system includes a powerful SAMAA AI Mistake Tracker that listens to student recitation, converts speech to text using offline Automatic Speech Recognition (ASR), compares transcribed text with authentic Quranic verses, and detects missed Ayah, skipped Ayah, repeated Ayah, and stuck points. For each detected mistake, the system highlights the error in Quranic text, provides correct recitation audio playback, and generates detailed mistake reports.

Students can view assigned tasks, record their recitation, receive instant AI powered feedback, view highlighted mistakes, listen to correct recitation, and track their personal progress through visual charts showing memorized versus remaining Quran portions and mistake trends over time. The accounting module enables administrators to track donations, manage expenses and generate financial reports.

QMMS requires a stable internet connection for all core functionalities including login, data synchronization, AI powered mistake detection, and report generation. The application uses Firebase cloud services for authentication, database storage, file storage, and real-time updates. This cloud-based architecture ensures data consistency across multiple devices, enables real time notifications, and provides scalable AI processing capabilities.

The system is developed using Flutter for cross platform mobile and desktop application development, Firebase Firestore for cloud database services, Firebase Authentication for secure user management, Cloud based Speech Recognition API for AI powered recitation analysis and Firebase Cloud Messaging for push notifications. These technologies ensure a responsive, secure, scalable and fully functional application with real time synchronization across all devices.

In conclusion, QMMS is designed to revolutionize Madrassa management by automating manual processes, providing AI powered recitation analysis and offering complete performance visibility. It promotes efficient memorization tracking, disciplined practice, data-driven decision making and balanced institutional growth. This project demonstrates practical implementation of software engineering concepts, cross platform mobile development, artificial intelligence, database systems and user centered design principles.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Dr. Muhammad Shoaib**” and our Co-Supervisor “**Mr. Arslan Sarwar**”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Amina Zafar

Ayesha Saddiqa

Dua Zafar

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
ASR	Automatic Speech Recognition
CURD	Create, Read, Update, Delete
DB	Database
ERD	Entity Relationship Diagram
FCM	Firebase Cloud Messaging
FR	Functional Requirement
GB	Gigabyte
GUI	Graphical User Interface
HCI	Human Computer Interaction
IDE	Integrated Development Environment
iOS	iPhone Operating System
MB	Megabyte
NFR	Non-Functional Requirement
OS	Operating System
PC	Personal Computer

PDF	Portable Document Format
QMMS	Quran Memorization Management System
RAM	Random Access Memory
REST	Representational State Transfer
SDK	Software Development Kit
SDD	Software Design Description
SDLC	Software Development Life Cycle
SQL	Structured Query Language
SRS	Software Requirement Specification
UI	User Interface
UX	User Experience
UML	Unified Modeling Language

Table of Contents

1.	Introduction	1
1.1	Project in Brief.....	1
1.2	Relevance of Course Modules	2
1.3	Project Background	3
1.4	Literature Review	4
1.5	Analysis from Literature Review	5
1.6	Methodology and Software Lifecycle for This Project.....	6
1.6.1	Rationale behind Selected Methodology	6
2.	Problem Definition	9
2.1	Problem Statement	10
2.2	Deliverables and Development Requirements.....	11
2.2.1	Deliverables.....	11
2.2.2	Development Requirements	12
2.3	Current System	13
3.	Requirement Analysis	16
3.1	Use Case Diagram	16
3.2	Detailed Use Cases	18
3.2.1	User Authentication.....	18
3.2.2	Assign Students to Teachers	19
3.2.3	View Student/Teacher Progress	20
3.2.4	Manage Donations/Expenses	21
3.2.5	Mark Attendance	23
3.2.6	Assign Task to Students	24
3.2.7	Samaa AI	25
3.2.8	View Assigned task.....	27
3.2.9	Record Recitation	28
3.3	Functional Requirements	29
3.3.1	Authenticate Users	29
3.3.2	Assign Students to Teachers	30
3.3.3	Mark Attendance	31
3.3.4	Assign Task to Students	32
3.3.5	SAMAA AI Mistake Tracker	33
3.3.6	View Progress	34
3.3.7	Manage Donations/Expenses	35
3.3.8	View Student/Teacher Progress	36
3.4	Non-Functional Requirements	37
3.4.1	Usability	37
3.4.2	Performance	37
3.4.3	Reliability & Availability	38
3.4.4	Security	38

3.4.5	Scalability	39
4.	Design and Architecture	41
4.1	System Architecture	41
4.1.1	Architectural Design	42
4.2	Data Representation	43
4.2.1	Data Dictionary	44
4.3	Process Flow Representation	51
4.4	Design Model	52
4.4.1	Class Diagram	53
4.4.2	Sequence Diagram	55
4.4.3	Process Flow Diagram	59
5.	Implementation	63
5.1	Algorithm	63
5.2	External APIs	68
6.	Testing and Evaluation	84
6.1	Manual Testing	84
6.1.1	System Testing	84
6.1.2	Unit Testing	85
6.1.3	Functional Testing	87
6.1.4	Integration Testing	89
6.2	Automated Testing	91
6.3	Tools Used	93
7.	Conclusion and Future Work	96
7.1	Conclusion	96
7.2	Future Work	96
8.	References	98

List of Figures

Figure 3.1: QMMS Use Case Diagram	17
Figure 4.1: QMMS Architecture Diagram	43
Figure 4.2: QMMS Entity Relationship Diagram	50
Figure 4.3: QMMS Class Diagram	54
Figure 4.4: Performance Analysis Sequence Diagram	55
Figure 4.5: Mistake Detection Sequence Diagram	56
Figure 4.6: Student Management Sequence Diagram	57
Figure 4.7: Manage Accounts (teacher, student profile)	58
Figure 4.8: QMMS Flow Diagram	60
Figure 4.9: Samaa AI Flow Diagram	61
Figure 5.1: Welcome Screen	70
Figure 5.2: SignIn Screen	71
Figure 5.3: Student Dashboard Screen	72
Figure 5.4: Manage Students Attendance	73
Figure 5.5: Memorization Screen	74
Figure 5.6: Teacher Dashboard Screen	75
Figure 5.7: Teachers, Students Performance Report(Downloaded) Screen	76
Figure 5.8: Course Management Screen	77
Figure 5.9: Admin Screen	78
Figure 5.10: Signup Screen	79
Figure 5.11: Forget Password Screen	80
Figure 5.12: Listening Correct Audio	81
Figure 5.13: Mistake History	82

List of Tables

Table 1.1: Related System Analysis with proposed project solution	5
Table 3.1: User Authentication	18
Table 3.2: Assign Students to Teachers	19
Table 3.3: View Student/Teacher Progress	21
Table 3.4: Manage Donations/Expenses	22
Table 3.5: Mark Attendance	23
Table 3.6: Assign Task to Students	24
Table 3.7: Samaa AI	25
Table 3.8: View Assigned Task	27
Table 3.9: Record recitation	28
Table 3.10: User Authentication	29
Table 3.11: Task Management	30
Table 3.12: Mark Attendance	32
Table 3.13: Assign task to Students	32
Table 3.14: Samaa AI Mistake Tracker	33
Table 3.15: View Progress	34
Table 3.16: Manage Donations/Expenses	35
Table 3.17: View Student/Teacher Progress	36
Table 4.1: QMMS Data Dictionary	45
Table 6.1: System Testing	84
Table 6.2: Unit Testing	86
Table 6.3: Functional Testing	87
Table 6.4: Integration Testing	90
Table 6.5: Automated Testing	91
Table 6.6: Testing Tools and Their Usage	93

Chapter 1

INTRODUCTION

1. Introduction

Quran Memorization Management System (QMMS) is a modern cross platform mobile and desktop application designed to simplify managing, tracking and improving Quran memorization (Hifz) in Madrassa. In today's digital age, traditional Madrassa often face challenges such as unrecorded student mistakes, difficulty tracking memorization progress, high teacher workload during Samaa (listening) sessions, and the frustration of using multiple disconnected manual systems. Constantly switching between physical notebooks for attendance, verbal task assignment, and manual mistake tracking adds unnecessary complexity and reduces educational quality. QMMS addresses this problem by combining essential Madrassa management features into a single, cohesive platform. The system offers role-based access for administrators, teachers and students within a unified application. Core modules include student enrollment, teacher assignment, three shift attendance management, task assignment with specific Surah and Ayah ranges, AI powered Samaa mistake tracker for recitation analysis, performance analytics and financial accounting for donations and expenses. Developed using Flutter for cross platform mobile and desktop development, with Firebase for backend cloud services including authentication, database, storage and real time synchronization. The system leverages AI powered speech recognition to automatically detect recitation mistakes including missed Ayah, skipped Ayah, repeated Ayah, and stuck points, providing instant feedback to students and detailed reports to teachers. By minimizing manual workload and merging attendance, task assignment, mistake tracking and financial management into a single intuitive platform, QMMS enhances Madrassa efficiency, reduces teacher stress and fosters better Quran memorization outcomes.

1.1 Project in Brief

Quran Memorization Management System (QMMS) is an intelligent cross platform application designed to help Madrassa administrators manage institutions, teachers track student progress and students improve their Quran memorization within a single platform. It allows administrators to enroll students, assign teachers and monitor overall performance, while teachers can mark attendance across three shifts, assign specific Surah and Ayah ranges as tasks, and review AI generated mistake reports. QMMS solves the problem of fragmented Madrassa management tools by integrating all essential features into one user friendly application.

The system also includes an AI powered Samaa mistake tracker that listens to student

recitation, converts speech to text using cloud based speech recognition, compares transcribed text with authentic Quranic verses, detects missed, skipped and repeated Ayah, highlights mistakes in Quranic text, provides correct recitation audio playback and generates detailed mistake reports. Additional features include three shift attendance management, task assignment with progress tracking, performance analytics, teacher evaluation metrics and accounting module for donation and expense tracking.

The application is developed for Android, iOS, and Windows using Flutter for the frontend and Firebase for backend and cloud services, following Object Oriented Design (OOD) and the Agile development lifecycle. The initial version focuses on core features like student or teacher management, three-shift attendance, task assignment, AI powered mistake tracking, performance reporting and accounting, while future enhancements include parent portal, multiple Qira'at support and advanced Tajweed error detection.

1.2 Relevance of Course Modules

The development of QMMS is closely related to the core courses studied in the Software Engineering (SE) program, as it applies both theoretical knowledge and practical development skills. The project strongly relates to Software Engineering, where concepts such as requirement gathering, Software Requirement Specification (SRS), Software Design Description (SDD), software development life cycle models, testing and project management are applied throughout the development process. It also involves Object Oriented Programming, where classes, objects, inheritance, encapsulation and modular programming techniques are used to build a structured and maintainable system for Users, Students, Teachers, Tasks, Attendance and Recitations.

The project is connected to Database Systems through the design and management of databases using Firebase Firestore for secure cloud storage of user accounts, student records, teacher assignments, attendance logs, tasks, recitation sessions, mistake reports, donations and expenses. Knowledge of Mobile and Desktop Application Development is utilized in building the cross-platform application using the Flutter framework, ensuring a responsive experience on Android, iOS and Windows devices with features like audio recording, media playback and real time data synchronization. Concepts of Human Computer Interaction(HCI) are applied to design a clean, intuitive and user-friendly interface that supports Arabic script, provides clear navigation for users with varying technical proficiency and ensures accessibility for all Madrassa users.

QMMS also relates to Artificial Intelligence, as cloud based speech recognition and string matching algorithms are integrated for recitation analysis, mistake detection, and automated feedback generation. In addition, Computer Networks concepts are used for Firebase cloud synchronization, API communication, real time data transfer, and push notifications for task assignments. Finally, Data Structures and Algorithms play an important role in efficient data handling, text comparison using Levenshtein distance algorithm, tokenization for word by word Quranic comparison, and overall system performance. Therefore, QMMS combines knowledge from multiple course modules into one complete and meaningful final year project.

1.3 Project Background

With the increasing demands of Quran memorization programs, Madrassa teachers often struggle to manage 20 to 30 students simultaneously during Samaa (listening) sessions. Most existing solutions focus on a single aspect such as basic attendance tracking or fee collection, forcing Madrassa to rely on multiple disconnected systems or entirely manual paper-based methods. This not only creates fragmentation but also reduces educational quality and increases teacher workload. The idea behind QMMS is to provide a unified Madrassa management platform that combines all essential features into one system, allowing administrators, teachers, and students to manage memorization activities more effectively and in a structured manner.

The project team conducted a survey of five local Madrassa in Sahiwal district with 127 respondents. Findings revealed that average students per teacher is 28, 100% of Madrassa use notebooks for manual mistake tracking, 0% have digital record maintenance, 82% of teachers reported feeling overwhelmed, 91% of students expressed desire for independent practice tools, 100% of administrators wanted data dashboards, and 67% reported donation tracking problems.

During a typical Samaa session, each student gets approximately 2-3 minutes of focused attention, teachers cannot note all mistakes for all students, repeated errors go unnoticed, no record exists for future reference, and students with similar mistakes cannot be grouped for focused correction.

QMMS works by integrating core Madrassa management components including student and teacher management, three-shift attendance, task assignment with specific Surah and Ayah ranges, AI powered Samaa module for automatic recitation mistake detection, performance

analytics for visual progress tracking and teacher evaluation and accounting module for donation and expense management. The system leverages modern cross-platform development technologies (Flutter) and cloud services (Firebase) to ensure real-time data synchronization, secure access and scalability. By combining these features, QMMS aims to simplify Madrassa management, improve memorization quality and enhance overall institutional efficiency.

1.4 Literature Review

Recent trends in Madrassa management and Quran memorization applications focus on AI based recitation analysis, gamification and all in one management solutions. However, most existing systems still lack a balanced integration of comprehensive management features, accurate mistake detection and user-friendly interfaces suitable for traditional Madrassa environments.

Tarteel.ai: Tarteel.ai is one of the leading AI powered Quran recitation platforms offering real time mistake detection with high accuracy and immediate feedback. However, it lacks built in Madrassa management features such as attendance tracking, teacher assignment, student grouping and performance reporting, limiting its use for institutional settings. It is also internet dependent with no offline functionality.

Quran Companion: Quran Companion helps individuals track their memorization progress and offers revision scheduling. While it provides good individual use features and progress tracking, it lacks AI powered recitation analysis, teacher-student workflow and institutional features needed for Madrassa environments.

Traditional Madrassa Software (Madrassa Tracker, Al-Hafiz): Traditional Madrassa management software focuses primarily on administrative tasks like fee collection, student registration and basic attendance. These systems completely miss the core educational need of recitation quality analysis and mistake tracking and often feature outdated interfaces that are not user friendly.

Manual Paper-Based System: The manual system remains the most common approach in traditional Madrassas where teachers maintain physical registers for attendance and use notebooks to manually note student mistakes. This system has no data analysis capability, no historical trend tracking, records are easily lost or damaged and there is no way to correlate attendance patterns with memorization performance.

The related systems provide a foundation in either AI based recitation analysis or basic Madrassa management, but there remains a significant gap in offering an integrated, intelligent and easy to use solution that combines comprehensive management features with accurate AI powered mistake detection for complete Madrassa memorization management.

1.5 Analysis from Literature Review

The analysis of existing systems highlights that no single application fully satisfies all Madrassa management and Quran memorization tracking needs. Tarteel.ai excels in AI mistake detection but lacks Madrassa management features. Traditional software handles administration but has no recitation quality analysis. Manual systems are familiar but provide no data tracking or analytics. Notion provides flexibility but becomes complex and lacks Quran specific features.

From the analysis given in Table 1.5, it is clear that Madrassas require a system that combines AI powered recitation analysis, comprehensive management features and user friendly interfaces. QMMS addresses these limitations by offering a unified platform that integrates student enrollment, teacher assignment, three shift attendance, task assignment, AI powered mistake tracking, performance analytics and financial accounting in a single application. Unlike existing systems, it focuses on ease of use, automation and educational quality enhancement, making it more suitable for Madrassa administrators, teachers and Hifz students.

Table 1.1: Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
Tarteel.ai	Lacks Madrassa management features (attendance, teacher assignment, student grouping)	QMMS will integrate both AI mistake detection and complete Madrassa management including three shift attendance and teacher assignment
Quran Companion	No AI mistake detection, no teacher student workflow	QMMS will offer AI powered SAMAA mistake tracking alongside teacher and student dashboards with task assignment

Traditional Madrassa Software	No recitation quality analysis, no AI capabilities, outdated interfaces	QMMS will provide modern cross platform interface with built in AI recitation analysis and real time synchronization
Manual Paper Based System	No data analysis, no historical tracking, records easily lost	QMMS will offer complete digital record keeping with cloud backup, trend analysis, and automated report generation

1.6 Methodology and Software Lifecycle for This Project

The development of QMMS follows the Object-Oriented Design (OOD) methodology, which emphasizes modularity, reusability and scalability. By modeling the system as a collection of interacting objects such as Users, Students, Teachers, Admins, Tasks, Attendance, Recitations, Mistakes, Donations and Expenses, OOD allows for a clear representation of real world entities and their behaviors. This approach simplifies maintenance, enables code reuse across different modules and supports future expansion, such as integrating new features like parent portal or multiple Qira'at support. Each module in QMMS, is designed as an independent object with defined attributes and methods, promoting separation of concerns and reducing system complexity.

For the software development process, QMMS follows the Agile methodology, which supports iterative and incremental development. Agile allows the team to deliver functional components in small cycles of 1 to 3 week sprints, gather continuous user feedback from Madrassa teachers and students, and adapt to evolving requirements. Combining OOD with Agile ensures a flexible, maintainable and quality focused development process, allowing QMMS to evolve efficiently while meeting Madrassa user expectations.

1.6.1 Rationale behind Selected Methodology

The selection of Object Oriented Design (OOD) for QMMS is based on the need to efficiently manage a system composed of multiple modules such as Users, Students, Teachers, Tasks, Attendance, Recitations, Mistakes, Donations and Expenses. OOD enables modularity, reusability and clear structure by representing real world entities as objects, making the system easier to develop, maintain and expand. It also supports encapsulation and abstraction, which reduce complexity and allow new features such as parent portal or multiple Qira'at

support to be added with minimal impact on existing components, making it more suitable than traditional structural approaches for a scalable application.

The adoption of the Agile software development lifecycle is justified by the project's need for flexibility and continuous improvement. Agile supports iterative development, allowing features like three shift attendance, task assignment and AI mistake tracking to be built, tested and refined in cycles with regular user feedback from Madrasa teachers. This ensures better quality, faster adaptation to changing requirements such as the addition of three shift attendance based on teacher feedback during development and a final product that closely aligns with Madrasa user needs, making Agile an ideal choice for QMMS.

Chapter 2

PROBLEM DEFINITION

2. Problem Definition

The Quran Memorization Management System (QMMS) provides a unified platform for Madrassa management and Quran memorization tracking, integrating student enrollment, teacher assignment, three shift attendance management, task assignment, AI powered SAMAA mistake tracking, performance analytics, and financial accounting. Administrators can add, edit, and delete student and teacher accounts, assign teachers to students, and monitor overall Madrassa performance. Teachers can mark attendance across three shifts, assign specific Surah and Ayah ranges as memorization tasks to students, and review AI generated mistake reports for each student's recitation. Students can view assigned tasks, record their recitation using device microphone, receive instant AI powered feedback, view highlighted mistakes in Quranic text, listen to correct recitation audio playback, track personal progress and view their attendance records. The AI powered SAMAA mistake tracker converts recitation speech to text using cloud based speech recognition, compares transcribed text with authentic Quranic verses, detects missed Ayah, skipped Ayah, repeated Ayah and stuck points, highlights mistakes with color coding, provides correct recitation audio playback, generates detailed mistake reports and stores session history for progress tracking.

Additional features include performance analytics for student progress tracking showing memorized versus remaining Juz, mistake trends over time, weak area identification for specific Surah and Ayat, teacher evaluation metrics including graduation rate and average completion time, and accounting module for donation and expense management with balance calculation and financial reporting. The main feature of this QMMS is Samaa module that helps the students in the memorization process. The system ensures secure user authentication through Firebase Authentication, cloud data synchronization through Firebase Firestore, push notifications through Firebase Cloud Messaging and privacy controls for student and teacher data. Quran Memorization Management System (QMMS) is implemented for Android, iOS and Windows platforms using Flutter for cross platform frontend development and Firebase for backend cloud services. The initial implementation focuses on core Madrassa management features including student and teacher management, three shift attendance, task assignment, AI powered mistake tracking, performance reporting and accounting, with future scalability planned for parent portal, multiple Qira'at support and advanced Tajweed error detection.

2.1 Problem Statement

Managing Quran memorization programs in traditional Madrasa has become increasingly challenging, with student records, attendance, task assignments and mistake tracking scattered across multiple manual systems. Existing apps provide partial solutions such as basic attendance tracking or individual Quran memorization apps, but they often lack intelligent features that adapt to Madrasa needs. Switching between physical notebooks for attendance, verbal task assignment, manual mistake tracking during Samaa sessions and separate financial records causes stress, confusion and frustration when student progress goes unrecorded due to poor organization. This fragmented experience wastes teacher time, affects memorization consistency, leaves student mistakes untracked and provides no data for evaluating teacher effectiveness.

Many existing applications provide partial solutions, but they rarely combine student enrollment, teacher assignment, three shift attendance, task assignment with specific Surah and Ayah ranges, AI powered recitation mistake detection, performance analytics, teacher evaluation and financial accounting within one cohesive system. A single teacher manages 20 to 30 students during Samaa sessions, making it impossible to focus on each student's mistakes properly. Students have no systematic way to identify their own recitation errors, cannot track which Surah or Ayat they are weak in and have no method to understand how to correct repeated mistakes. Administrators have no data driven way to evaluate teacher performance, no visibility into overall institutional effectiveness and no financial transparency for donations and expenses tracking.

QMMS addresses this problem by offering an all in one intelligent Madrasa management platform that integrates student enrollment, teacher assignment, three shift attendance, task assignment, AI-powered Samaa mistake tracking, performance analytics, teacher evaluation and financial accounting in a unified space. It enhances memorization quality, reduces teacher workload and helps administrators maintain balance in institutional operations. While apps like Tarteel.ai focus only on AI recitation analysis with no Madrasa management features, Quran Companion focuses only on individual memorization with no teacher-student workflow and traditional software handles only administration with no recitation quality analysis or AI capabilities.

Developing this system enhances understanding of cross platform mobile and desktop application development, cloud synchronization, AI integration, database management and

user centered design principles. It also strengthens technical expertise in Flutter, Firebase, speech recognition APIs, database management, responsive UI/UX development and data analytics. This project provides an opportunity to apply theoretical knowledge in a practical environment, promoting a deeper understanding of the software development lifecycle (SDLC) and preal-world problem solving while contributing an innovative, meaningful solution to modern Madrassa management and Quran memorization tracking challenges.

2.2 Deliverables and Development Requirements

The main deliverable of QMMS will be a fully functional cross-platform application developed using Flutter for Android, iOS, and Windows, providing an all-in-one platform for Madrassa management. The development requires Android Studio and VS Code, Flutter SDK, Firebase services (Authentication, Firestore, Storage, FCM), and Figma for UI/UX prototyping with Arabic script support.

2.2.1 Deliverables

The main deliverable of the QMMS project will be a fully functional cross platform application developed using Flutter for Android, iOS and Windows platforms. The application will provide an all in one platform for Madrassa management including student enrollment, teacher assignment, three shift attendance management, task assignment with specific Surah and Ayah ranges, AI powered Samaa mistake tracking, performance analytics, teacher evaluation and financial accounting. Administrators, teachers and students will be able to manage their respective activities through role based personalized dashboards.

Another important deliverable, the system will include a secure authentication module that allows users to register, log in with role based access (Admin, Teacher, Student), reset passwords and manage profiles. Firebase Authentication will be used to ensure secure access and data privacy. This module will also support user session management for seamless login experiences across mobile and desktop platforms.

QMMS will deliver an integrated module for three shift attendance management. Teachers will be able to mark attendance for assigned students, select dates for attendance records and view attendance summaries. Administrators will be able to view attendance reports filtered by date, teacher or shift.

The system will include a comprehensive task assignment module allowing teachers to assign specific Surah and Ayah ranges to students as memorization tasks. The module will include

Surah selection, Ayah range validation (ensuring $\text{start} \leq \text{end}$ and within Surah limits), task status tracking (pending, in-progress, completed), and task history per student.

Project will deliver an AI powered Samaa mistake tracker that listens to student recitation, converts speech to Arabic text using cloud based speech recognition, fetches correct Quranic text for assigned task, compares transcribed text with Quranic text using string matching algorithms, detects missed Ayah, skipped Ayah, repeated Ayah and stuck points, highlights mistakes in Quranic text with color coding, provides correct recitation audio playback for each mistake, generates detailed mistake reports and stores recitation sessions for history tracking.

QMMS will include performance analytics for student progress tracking showing memorized versus remaining Juzz through visual charts, mistake trends over time with line charts, weak area identification for specific Surah and Ayat and teacher evaluation metrics including graduation rate and average completion time. The accounting module will allow administrators to record donations with donor name, amount, date and payment mode, record expenses with description, amount, date and category, calculate total donations, total expenses and balance and generate financial reports by date range.

The project will also deliver complete academic and technical documentation including the Software Requirement Specification (SRS), Software Design Description (SDD), testing reports, user manual, presentation slides and final report. These documents will support implementation, maintenance, and evaluation of the system.

2.2.2 Development Requirements

The development of QMMS requires Android Studio and Visual Studio Code as the main IDEs, Flutter SDK for cross platform mobile and desktop application development, and Firebase services for backend operations including Firebase Authentication, Firebase Firestore database, Firebase Storage for audio files, and Firebase Cloud Messaging for push notifications. UI/UX design tools such as Figma will be used for interface prototyping with Arabic script support.

The project also requires computers with sufficient processing power for cross platform development, Android and iOS mobile devices for testing, Windows desktop for desktop application testing, and a stable internet connection required for cloud synchronization, authentication, speech recognition services, and real-time data transfer. In addition,

successful development depends on effective teamwork, task division and supervision throughout the project lifecycle. Regular progress reviews and sprint retrospectives will be conducted to ensure timely delivery, quality assurance and continuous improvement in alignment with Agile development practices.

2.3 Current System

At present, Madrassa depend on multiple separate manual and digital systems to manage memorization related activities. Student enrollment and record keeping is often handled through physical registers or basic spreadsheets, attendance through paper registers, task assignment through verbal communication during class, mistake tracking through manual notebooks during Samaa sessions, performance tracking through teacher memory or informal notes and financial management through separate ledgers or basic accounting software. Since these systems operate independently, teachers must frequently switch between notebooks, verbal communication and memory to manage their daily routines, which increases complexity and reduces efficiency.

Most current Madrassa management solutions focus only on one specific feature and do not provide a complete memorization management solution. Tarteel.ai focuses on AI recitation analysis but lacks Madrassa management features such as attendance tracking, teacher assignment, student grouping, and performance reporting. Quran Companion focuses on individual memorization tracking but lacks AI mistake detection and teacher-student workflow. Traditional Madrassa software like Madrassa Tracker and Al-Hafiz handle student registration, fee collection, and basic attendance but completely miss the core educational need of recitation quality analysis and mistake tracking. The manual paper-based system remains the most common approach where teachers maintain physical registers for attendance and use notebooks to manually note student mistakes. Many systems lack integrated AI powered recitation analysis, automated mistake tracking, performance analytics or teacher evaluation metrics, forcing teachers to rely on manual observation and memory for guidance and assessment. In addition, several Madrassa management applications provide limited or no collaboration features between teachers and students, making it difficult for students to receive timely feedback on their recitation mistakes or for teachers to track improvement over time.

The major weakness of the current system is fragmentation. Student records, attendance, task assignments, mistake tracking, performance data and financial records are distributed across

multiple notebooks, registers, verbal communication and separate applications, resulting in wasted teacher time, unnoticed recitation mistakes, inconsistent student progress tracking and poor institutional management. Furthermore, storing student data across different notebooks with no backup creates security concerns of lost or damaged records and reduces administrator control over institutional information. Teachers have no systematic way to record which student made which mistake, how many times a mistake was repeated, whether a student has improved over time, or how teacher performance compares across different instructors.

Therefore, the current system does not fully meet the needs of modern Madrassas. QMMS is proposed to overcome these limitations by offering one integrated, intelligent, and user friendly Madrassa management and Quran memorization tracking platform.

Chapter 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

The development of QMMS follows the Object-Oriented Design (OOD) methodology, which emphasizes modularity, reusability, and scalability. By modeling the system as a collection of interacting objects such as Users, Students, Teachers, Admins, Tasks, Attendance, Recitations, Mistakes, Donations and Expenses OOD allows for a clear representation of real-world entities and their behaviors. This approach simplifies maintenance, enables code reuse across different modules and supports future expansion, such as integrating new features like parent portal or multiple Qira'at support. Each module in QMMS, including student management, teacher assignment, three shift attendance, task assignment, AI mistake tracking, performance reporting and accounting, is designed as an independent object with defined attributes and methods, promoting separation of concerns and reducing system complexity.

For the software development process, QMMS follows the Agile methodology, which supports iterative and incremental development. Agile allows the team to deliver functional components in small cycles, gather continuous user feedback from Madrasa teachers and students and adapt to evolving requirements. This approach is particularly suitable for QMMS, as it ensures that core modules like three shift attendance, AI powered Samaa mistake tracking and performance reporting can be tested, refined and improved before full scale deployment. Combining OOD with Agile ensures a flexible, maintainable and quality focused development process, allowing QMMS to evolve efficiently while meeting Madrasa user expectations.

3.1 Use Case Diagram

The use case diagram for the QMMS system shows all the basic tasks that the users will perform in the app including Admin view student and teacher progress, assigning teachers to students, and managing finances. Teachers marking attendance across three shifts, assigning tasks with Surah and Ayah ranges and reviewing student progress. Students viewing assigned tasks, recording recitation, viewing progress and viewing attendance. Figure 3.1 also includes the Samaa AI Mistake Tracker module. Extended and included use cases like password recovery, shift attendance, submit audio for analysis and performance analytics enhance the overall user experience.

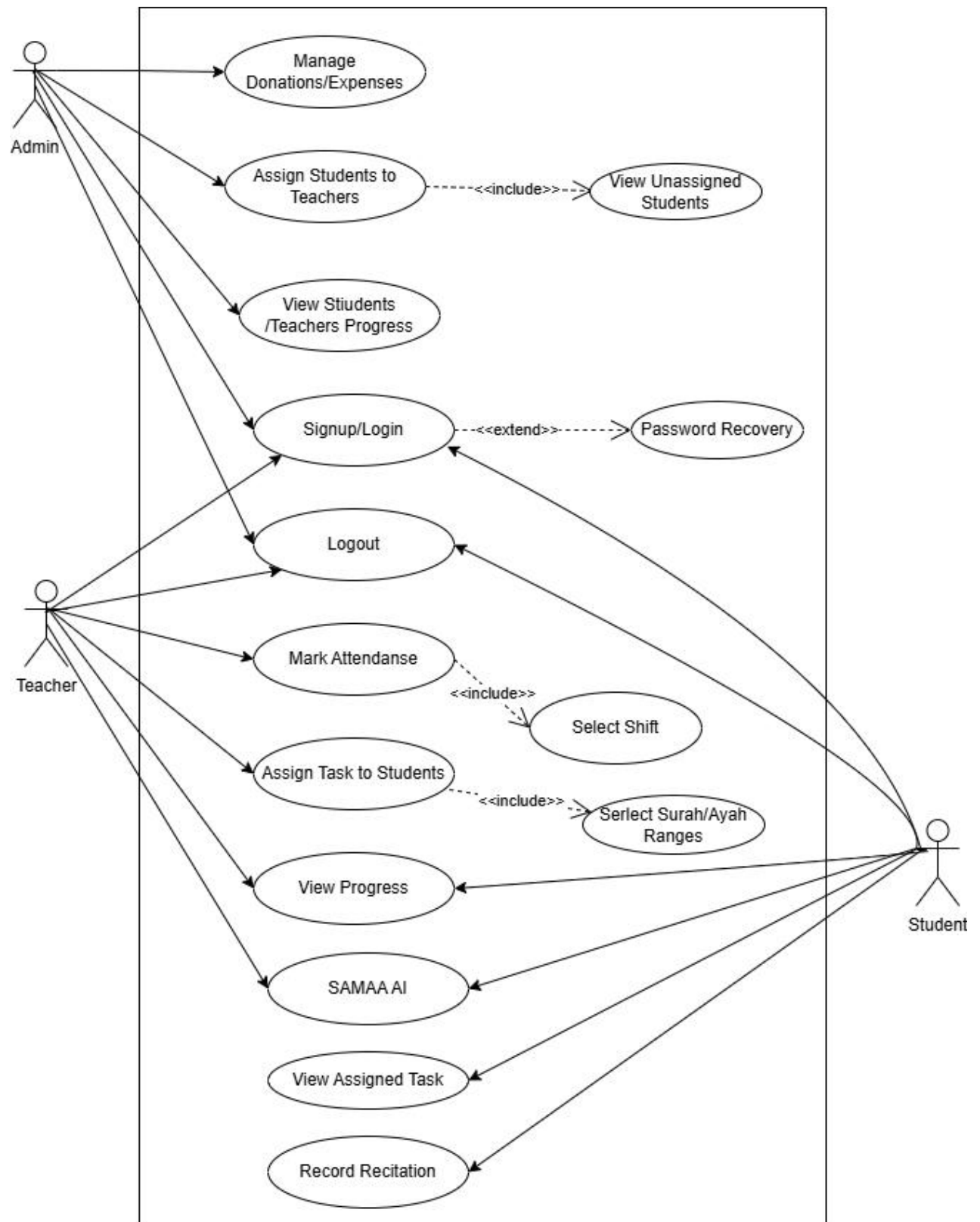


Figure 3.1: QMMS Use Case Diagram

3.2 Detailed Use Cases

This section presents the detailed use cases of the QMMS system. Each use case describes a specific interaction between the user and the system, including the actors involved, preconditions, normal flow, alternative flow, postconditions, and business rules. These use cases help in understanding how different modules of the application will function and how users will interact with the system.

3.2.1 User Authentication

In Table 3.1, User Authentication allows users to securely register, log in, and manage their profiles. It protects personal data through encrypted credentials and secure verification methods. The module also maintains user sessions for smooth access across multiple devices. It ensures that only authorized users can access private Madrasa data and personalized features. This process builds trust, security, and convenience for users.

Table 3.1: User Authentication

Use Case ID	UC1
Use Case Name	User Authentication
Actors	Primary: Admin, Teacher, Student Secondary: Authentication Service, Database
Description	A user registers or logs in to the the QMMS app to access personalized features such as student management, attendance marking, task assignment, and mistake tracking based on their role.
Trigger	User opens the app and selects “Login” or “Sign Up.”
Preconditions	PRE-1. The app must be installed and running. PRE-2. User has a stable internet connection.
Postconditions	POST-1. User account is verified and stored in the database. POST-2. User is redirected to the role specific dashboard.

Normal Flow	1. User opens the app and selects “Sign Up” or “Login.” 2. System displays authentication options (email/password). 3. User enters credentials and submits. 4. System validates credentials with the authentication service. 5. On success, system fetches user data and role from the database. 6. System displays personalized role based dashboard.
Alternative Flows	1.1.Invalid credentials entered → System displays “Invalid credentials” message and allows retry. 1.2.Forgot password → User selects “Forgot Password,” enters email, and resets via email link.
Exceptions	Authentication service unavailable → System notifies user and retries after delay.
Business Rules	BR-1 Passwords must be at least 8 characters long. BR-2 Email must be unique. BR-3 Each user must have a defined role (Admin, Teacher, Student)
Assumptions	Assume the majority of users register via email/password.

3.2.2 Assign Students to Teachers

In Table 3.2, this use case enables Admin to assign a specific teacher to a student for mentorship and progress tracking. The assignment links the student to the teacher for all future interactions including attendance marking, task assignment, and mistake report review. This ensures proper student teacher relationships within the Madrassa.

Table 3.2: Assign Students to Teachers

Use Case ID	UC2
Use Case Name	Assign Students to Teachers

Actors	Primary: Admin Secondary: Database
Description	Admin assigns a specific teacher to a student for mentorship and progress tracking.
Trigger	Admin selects "Assign Students to Teachers" from dashboard.
Preconditions	PRE-1. Admin must be logged in.
Postconditions	POST-1. Student is assigned to teacher in the database. POST-2. Student appears in teacher's assigned student list.
Normal Flow	1. Admin selects "Assign Students to Teachers." 2. System displays list of unassigned students. 3. Admin selects a student. 4. System displays list of available teachers. 5. Admin selects a teacher. 6. System confirms assignment and updates database. 7. Student appears in teacher's dashboard.
Alternative Flows	2.1. Admin reassigns student to different teacher → System updates assignment.
Exceptions	Assignment fails due to network error → System shows retry option.
Business Rules	BR-1 Student can be assigned to only one teacher at a time. BR-2 Teacher must have active status.
Assumptions	Assume each student is assigned to exactly one teacher.

3.2.3 View Student/Teacher Progress

In Table 3.3, this use case enables Admin to monitor overall Madrassa performance by viewing progress of all students and teachers. The system displays key metrics including

student memorization progress, teacher effectiveness, and overall institutional performance.

Table 3.3: View Student/Teacher Progress

Use Case ID	UC3
Use Case Name	View student/Teacher Progress
Actors	Primary: Admin Secondary: Performance Module, Database
Description	Admin views overall student progress and teacher performance metrics.
Trigger	Admin selects "View Progress" from dashboard.
Preconditions	PRE-1. Admin must be logged in. PRE-2. Student and teacher data exists in the system.
Postconditions	POST-1. Progress reports are generated and displayed.
Normal Flow	1. Admin selects "View Progress." 2. System queries all student and teacher data. 3. System calculates overall memorization progress. 4. System calculates teacher performance metrics. 5. System displays progress dashboard.
Exceptions	Data fetch fails → System displays "Unable to load progress" message.
Business Rules	BR-1 Progress is calculated per student and per teacher. BR-2 Reports are generated in real-time.
Assumptions	Assume sufficient data exists for meaningful reports.

3.2.4 Manage Donations/Expenses

This use case enables Admin to record and manage donations received and expenses incurred by the Madrassa. Donation records include donor name, amount, date and mode of payment. Expense records include description, amount, date and category. In Table 3.4, the system tracks all financial transactions and provides summaries for reporting. The accounting module ensures complete financial transparency by maintaining a detailed ledger of all incoming and outgoing funds. Admin can also filter and search financial records by date

range, category or donor name for quick reference.

Table 3.4: Manage Donations/Expenses

Use Case ID	UC4
Use Case Name	Manage Donations/Expenses
Actors	Primary: Admin Secondary: Accounting Module, Database
Description	Admin records donations and expenses for financial management.
Trigger	Admin selects "Manage Finances" from dashboard.
Preconditions	PRE-1. Admin must be logged in.
Postconditions	POST-1. Financial records are stored in the database. POST-2. Total donations, expenses, and balance are updated.
Normal Flow	1. Admin selects "Manage Donations/Expenses." 2. System displays options for Donations and Expenses. 3. Admin selects "Add Donation" or "Add Expense." 4. System prompts for relevant details. 5. Admin fills details and clicks "Save." 6. System stores record and updates totals. 7. Record appears in respective list.
Alternative Flows	4.1. Admin edits existing record → Updates fields and saves. 4.2. Admin deletes record → System confirms deletion and removes record.
Exceptions	Save fails due to network error → System shows retry option.

Business Rules	BR-1 Donation amount must be positive. BR-2 Expense amount must be positive. BR-3 Date cannot be in future.
Assumptions	Assume financial records are maintained in local currency.

3.2.5 Mark Attendance

In Table 3.5, this use case enables teachers to mark student attendance across three shifts. Teachers can select the date and shift, mark students as present or absent and save attendance records. The system stores attendance with date, shift, student ID and status for future reference and reporting.

Table 3.5: Mark Attendance

Use Case ID	UC5
Use Case Name	Mark Attendance
Actors	Primary: Teacher Secondary: Attendance Module, Database
Description	Teacher marks student attendance for three shifts.
Trigger	Teacher selects "Mark Attendance" from dashboard.
Preconditions	PRE-1. Teacher must be logged in. PRE-2. Teacher has assigned students.
Postconditions	POST-1. Attendance records are stored in the database with date, shift, and status. POST-2. Attendance summary is updated.

Normal Flow	1. Teacher selects "Select Shift." 2. System displays shift options. 3. Teacher selects a shift. 4. System applies shift and displays students.
Exceptions	Shift selection fails → System displays error message.
Business Rules	BR-1 Only one shift can be selected at a time. BR-2 Shift cannot be changed after attendance is saved.
Assumptions	Assume each shift corresponds to a specific time slot.

3.2.6 Assign Task to Students

In Table 3.6, this use case enables teachers to assign specific Surah and Ayah ranges to students as memorization tasks. Teachers select the student, Surah, Ayah start and Ayah end. The system validates the Ayah range, stores the task, and displays it on the student's dashboard. This feature enables structured memorization planning and progress tracking.

Table 3.6: Assign Task to Students

Use Case ID	UC6
Use Case Name	Assign Task to Students
Actors	Primary: Teacher Secondary: Task Module, Database
Description	Teacher assigns specific Surah and Ayah range to student as memorization task.
Trigger	Teacher selects "Assign Task" from dashboard.
Preconditions	PRE-1. Teacher must be logged in. PRE-2. Teacher has assigned students.
Postconditions	POST-1. Task is stored in the database with Surah, Ayah range, and status. POST-2. Task appears on student's dashboard.

Normal Flow	1. Teacher selects "Assign Task." 2. System displays list of assigned students. 3. Teacher selects a student. 4. System displays Surah selection list. 5. Teacher selects Surah. 6. Teacher enters Ayah start and Ayah end. 7. System validates Ayah range (start $\leq$ end, within Surah limits). 8. Teacher clicks "Assign Task." 9. System stores task with status "Pending" and confirms assignment.
Alternative Flows	6.1. Invalid Ayah range $\rightarrow$ System displays error message. 6.2. Teacher edits existing task $\rightarrow$ Updates fields and saves.
Exceptions	Task save fails due to network error $\rightarrow$ System shows retry option.
Business Rules	BR-1 Ayah start must be $\leq$ Ayah end. BR-2 Ayah end must not exceed total Ayah in selected Surah. BR-3 Each task has status (Pending, In-Progress, Completed).
Assumptions	Assume Quran database contains all Surah names and Ayah counts.

3.2.7 Samaa AI

In Table 3.7, the Samaa AI is the core intelligent module of QMMS. It listens to student recitation, converts speech to Arabic text using cloud based speech recognition, compares transcribed text with authentic Quranic verses, detects missed Ayah, skipped Ayah, repeated Ayah, and stuck points, highlights mistakes in Quranic text , provides correct recitation audio playback, generates detailed mistake reports and stores session history. This feature reduces teacher workload and provides instant feedback to students.

Table 3.7: Samaa AI

Use Case ID	UC7
Use Case Name	Samaa AI

Actors	Primary: Teacher, Student Secondary: ASR Service, Comparison Engine, Database
Description	AI powered module that listens to recitation, converts to text, compares with Quran, detects and highlights mistakes, provides correct audio.
Trigger	Student selects "Start Recitation" or Teacher selects "Review Recitation."
Preconditions	PRE-1. User must be logged in. PRE-2. Student has assigned task. PRE-3. Microphone is available and permissions granted.
Postconditions	POST-1. Recitation is analyzed and mistakes are detected. POST-2. Mistake report is generated and stored. POST-3. Student progress is updated.
Normal Flow	1. User initiates Samaa AI module. 2. Student records recitation using device microphone. 3. System sends audio to cloud ASR service. 4. ASR converts speech to Arabic text. 5. System fetches correct Quranic text for task. 6. System normalizes both texts (removes diacritics). 7. System compares texts using string matching algorithm. 8. System detects missed Ayah, skipped Ayah, repeated Ayah. 9. System highlights mistakes in Quranic text . 10. System provides correct recitation audio playback for each mistake. 11. System generates detailed mistake report. 12. System stores session and updates student progress.
Alternative Flows	7.1. Student pauses and resumes recording. 7.2. No audio detected → System displays "No recitation detected" message. 7.3. Student can listen to correct audio for each mistake.
Exceptions	ASR service unavailable → System displays "Speech recognition unavailable" message.

Business Rules	BR-1 Recording cannot exceed 2 minutes per session. BR-2 Each mistake is categorized as Missed, Skipped, or Repeated. BR-3 Progress updates after each completed session.
Assumptions	Assume stable internet connection for cloud ASR service.

3.2.8 View Assigned task

In Table 3.8, this use case enables students to view the tasks assigned by their teachers. The task display includes the Surah name and Ayah range that the students are required to memorize. This feature helps students understand what they need to practice and allows them to track their memorization assignments more effectively.

Table 3.8: View Assigned Task

Use Case ID	UC8
Use Case Name	View Assigned Task
Actors	Primary: Student Secondary: Task Module, Database
Description	Student views tasks assigned by teacher including Surah name and Ayah range.
Trigger	Student selects "My Tasks" from dashboard.
Preconditions	PRE-1. Student must be logged in. PRE-2. Teacher has assigned at least one task.
Postconditions	POST-1. Assigned tasks are displayed to student.
Normal Flow	1. Student selects "My Tasks." 2. System queries database for tasks assigned to student. 3. System displays list of tasks with Surah name, Ayah range and status. 4. Student selects a task to view details. 5. System displays complete task details.
Alternative Flows	No tasks assigned → System displays "No tasks assigned yet" message.
Exceptions	Task fetch fails → System displays "Unable to load tasks" message.

Business Rules	BR-1 Only tasks assigned to the student are shown. BR-2 Tasks are sorted by assignment date.
Assumptions	Assume teacher assigns tasks regularly.

3.2.9 Record Recitation

This use case enables students to record their Quran recitation using the device microphone. In Table 3.9, this use case illustrates how students can start, pause, stop, and submit their recitation for AI analysis. The recorded audio is securely uploaded to the cloud, where it is processed by the Samaa AI module to analyze the recitation and generate accurate feedback on the student's performance.

Table 3.9: Record recitation

Use Case ID	UC9
Use Case Name	Record recitation
Actors	Primary: Student Secondary: Audio Recorder, Storage
Description	Student records Quran recitation using device microphone for AI analysis.
Trigger	Student selects "Start Recitation" from task details.
Preconditions	PRE-1. Student must be logged in. PRE-2. Student has assigned task. PRE-3. Microphone permissions are granted.
Postconditions	POST-1. Recitation audio is recorded. POST-2. Audio is uploaded for analysis.
Normal Flow	1. Student selects assigned task and clicks "Start Recitation." 2. System displays Surah and Ayah range for recitation. 3. Student clicks "Start Recording." 4. System starts recording using microphone. 5. Student recites the assigned verses. 6. Student clicks "Stop Recording." 7. System prompts "Submit for Analysis?"

	8. Student clicks "Submit." 9. System uploads audio for SAMAA AI analysis.
Alternative Flows	9.1. Student pauses and resumes recording. 9.2. Student cancels and re-records
Exceptions	Microphone not available → System displays error message.
Business Rules	BR-1 Audio file must be in supported format (MP3/WAV).
Assumptions	Assume device has sufficient storage space for recording.

3.3 Functional Requirements

Functional requirements define the core operations and services that QMMS must perform. They describe what the system should do in response to user actions. These requirements ensure that all important features are implemented correctly. They also serve as a foundation for development, testing, and system validation.

3.3.1 Authenticate Users

The system shall allow users to register, log in with role-based access, and reset their passwords securely. Users are assigned roles such as Admin, Teacher, and Student, each with appropriate access controls. Incorrect credentials trigger an error message, prompting users to try again. In Table 3.10, the login process is illustrated, where users enter their registered email address and password, and the system authenticates the credentials using Firebase Authentication. Upon successful authentication, the system retrieves the user's role from the database and loads the appropriate dashboard.

Table 3.10: User Authentication

Field	FR1
Title	User Authentication

Requirement	• FR1.1: Users must be able to sign up using email/password with role selection (Admin, Teacher, Student) • FR1.2: System must validate credentials before granting access. • FR1.3: Passwords must be stored securely (hashed and salted). • FR1.4: Users should be able to reset forgotten passwords via email verification. • FR1.5: Only authenticated users can access personalized features based on their role. • FR1.6: System must load role specific dashboard after successful login.
Source	Module 1 – User Authentication
Rationale	Ensure data privacy and secure access for each user.
Business Rule	Each email must be unique; passwords must meet defined complexity standards.
Dependencies	None
Priority	High

3.3.2 Assign Students to Teachers

In Table 3.11, the system shall allow the admin to assign teachers to students for effective mentorship and progress tracking. The admin can view a complete list of unassigned students who have not yet been assigned to any teacher, along with their enrollment dates and current progress status. The system displays a comprehensive list of available teachers, including their specialization and current student workload, enabling the admin to make informed assignment decisions based on teacher capacity and expertise. Once the assignment is completed, the student is linked to the assigned teacher for all future interactions, including attendance marking and task assignments.

Table 3.11: Task Management

Field	FR-2
Title	Assign Students to Teachers

Requirement	• FR2.1: Admin must be able to assign a teacher to a student. • FR2.2: System must display list of unassigned students. • FR2.3: System must display list of available teachers. • FR2.4: System must confirm assignment before saving. • FR2.5: Student must appear in teacher's assigned student list after assignment.
Source	Module 2 – Teacher Assignment
Rationale	Enables proper student-teacher relationships for effective memorization tracking.
Business Rule	Student can be assigned to only one teacher at a time; teacher must have active status.
Dependencies	FR1 (Authenticate Users)
Priority	High

3.3.3 Mark Attendance

The system shall support three shift attendance marking. Teachers can mark attendance for assigned students, select dates and view attendance summaries. The system provides a user-friendly interface for teachers to efficiently record attendance for all assigned students within minutes. Teachers can mark individual students as Present or Absent, or use bulk actions like "Mark All Present" and "Mark All Absent" for quick recording. The system validates that attendance can only be marked for current or past dates, preventing future date entries. Each student can have only one attendance record per date per shift, ensuring data integrity and preventing duplicate entries. The attendance module also allows teachers to view historical attendance records, filter by date range, and generate attendance reports for administrative review. Teachers can identify students with low attendance rates and take corrective action. Administrators can view attendance summaries across all teachers and shifts, filter by date, teacher, or shift. In Table 3.12, the attendance marking is shown as the functional requirement of the system. It shows how teacher mark attendance of the student and what steps are followed during attendance marking.

Table 3.12: Mark Attendance

Field	FR3
Title	Mark Attendance
Requirement	• FR3.1: System must support three shifts. • FR3.2: Teacher must be able to mark attendance for assigned students. • FR3.3: Teacher must be able to select date for attendance. • FR3.4: System must store attendance records with student ID, date, shift, status. • FR3.5: Admin must be able to view attendance summaries. • FR3.6: System must generate attendance reports filtered by date, teacher, shift.
Source	Module – 3 Attendance Management
Rationale	Enables teachers to efficiently record student attendance across multiple shifts.
Business Rule	Each student can have only one attendance record per date per shift.
Dependencies	FR1 (Authenticate Users), FR2 (Assign Students to Teachers)
Priority	High

3.3.4 Assign Task to Students

In Table 3.13, the system shall allow teachers to assign specific Surah and Ayah ranges to students as memorization tasks. The system validates the Ayah ranges, tracks the task status, and displays the assigned tasks on the students' dashboards. It ensures that the starting Ayah is less than or equal to the ending Ayah and that the ending Ayah does not exceed the total number of Ayahs in the selected Surah. If an invalid range is entered, the system displays a clear error message, guiding teachers to correct their input before assigning the task.

Table 3.13: Assign task to Students

Field	FR4
Title	Assign Task to Students

Requirement	• FR4.1: Teacher must be able to assign tasks to students. • FR4.2: Task must include Surah number and Ayah range. • FR4.3: System must validate Ayah range (start $\leq$ end, within Surah limits). • FR4.4: System must track task status (pending, in-progress, completed). • FR4.5: Student must be able to view assigned tasks. • FR4.6: Teacher must be able to view task completion status. • FR4.7: System must store task assignment date.
Source	Module 4 – Task Assignment
Rationale	Enables structured memorization planning and progress tracking.
Business Rule	Ayah start must be $\leq$ Ayah end; Ayah end must not exceed total Ayah in selected Surah.
Dependencies	FR1 (Authenticate Users), FR2 (Assign Students to Teachers)
Priority	High

3.3.5 Samaa AI Mistake Tracker

In Table 3.14, the system provides AI-powered recitation analysis by converting speech into text, comparing it with the corresponding Quranic text, detecting recitation mistakes, highlighting errors, and generating detailed reports with correct audio playback. This feature enables students to improve their recitation through instant AI-based feedback

Table 3.14: SAMAA AI Mistake Tracker

Field	FR5
Title	SAMAA AI Mistake Tracker
Requirement	FR5.1: System must allow recording of student recitation via microphone.

	• FR5.2: System must convert speech to Arabic text using cloud-based ASR. • FR5.3: System must fetch correct Quranic text for assigned task. • FR5.4: System must normalize text for comparison. • FR5.5: System must compare transcribed text with Quranic text. • FR5.6: System must detect missed Ayah, skipped Ayah, repeated Ayah. • FR5.7: System must highlight mistakes in Quranic text. • FR5.8: System must provide correct recitation audio playback. • FR5.9: System must generate detailed mistake report. • FR5.10: System must store recitation session for history.
Source	Module 5 – SAMAA AI Module
Rationale	Reduces teacher workload and provides instant feedback to students.
Business Rule	Recording cannot exceed 2 minutes per session; each mistake categorized as Missed/Skipped/Repeated.
Dependencies	FR1 (Authenticate Users), FR4 (Assign Task to Students)
Priority	Highest

3.3.6 View Progress

The system shall allow students and teachers to view memorization progress including Juz completed, mistake trends, weak areas, and overall performance metrics. In Table 3.15, the system tracks mistake patterns over time. eachers can access progress reports for all assigned students, with the ability to filter by individual student, date range, or specific Surah.

Table 3.15: View Progress

Field	FR6
Title	View Progress

Requirement	• FR6.1: System must generate student progress reports. • FR6.2: System must display memorized vs remaining Juz. • FR6.3: System must show mistake trends over time. • FR6.4: System must identify weak areas (specific Surahs/Ayat). • FR6.5: Teachers must be able to view progress of assigned students. • FR6.6: Students must be able to view personal progress.
Source	Module 7 – Performance Reporting
Rationale	Enables data driven decision making for student improvement.
Business Rule	Progress is calculated per student; Juz count based on completed tasks.
Dependencies	FR1 (Authenticate Users), FR4 (Assign Task to Students), FR5 (Samma AI)
Priority	Medium

3.3.7 Manage Donations/Expenses

In Table 3.16, the system shall allow Admin to record donations and expenses, calculate totals, and generate financial reports for complete financial management.

Table 3.16: Manage Donations/Expenses

Field	FR7
Title	Manage Donations/Expenses
Requirement	• FR7.1: Admin must be able to record donations with donor name, amount, date, payment mode. • FR7.2: Admin must be able to record expenses with description, amount, date, category. • FR7.3: System must calculate total donations, total expenses, balance.

	- FR7.4: System must generate financial reports by date range. - FR7.5: System must allow editing and deleting financial records.
Source	Module 8 – Accounting Module
Rationale	Enables financial transparency and accountability.
Business Rule	Donation and expense amounts must be positive; date cannot be in future.
Dependencies	FR1 (Authenticate Users)
Priority	Medium

3.3.8 View Student/Teacher Progress

The system shall enable admin to view the teachers and students progress. Admin measure progress on the bases of Quran completion. In Table 3.17, the admin steps to view the teachers and students progress is explained.

Table 3.17: View Student/Teacher Progress

Field	FR8
Title	View Student/Teacher Progress
Requirement	- FR8.1: Admin must be able to view all student progress across all teachers. - FR8.2: System must calculate teacher graduation rate. - FR8.3: System must calculate average completion time per teacher. - FR8.4: System must display teacher performance metrics. - FR8.5: Admin must be able to compare teacher performance.
Source	Module 9 – Performance Reporting
Rationale	Enables institutional performance monitoring and teacher evaluation.

Business Rule	Performance is calculated per student and per teacher.
Dependencies	FR1 (Authenticate Users), FR6 (View Progress)
Priority	Medium

3.4 Non-Functional Requirements

This section specifies the non-functional (quality) requirements for QMMS. These requirements ensure that the application not only meets functional needs but also maintains high usability, reliability, security and scalability standards.

3.4.1 Usability

Usability requirements define how easily users can learn and interact with the system. Since QMMS is targeted at Madrassa administrators, teachers and students with varying technical proficiency, it emphasizes a clean, intuitive and accessible design to reduce cognitive load and enhance engagement.

- **USE-1:** The system shall provide an intuitive and user interface that enables new users to understand and navigate major features (Student Management, Attendance, Task Assignment, Samaa AI) within 5 minutes of first use.
- **USE-2:** The system shall allow users to perform primary actions (mark attendance, assign task, record recitation) within three clicks or less.
- **USE-3:** The system shall display clear error messages for invalid inputs, ensuring users can recover from mistakes without external help.
- **USE-4:** The system shall support Arabic script rendering correctly across all interfaces.
- **USE-5:** The system shall provide clear navigation with role specific menus (Admin/Teacher/Student).

3.4.2 Performance

Performance requirements specify how well the system operates at both peak and typical usage levels. QMMS's responsive design guarantees low latency and synchronization.

- **PER-1:** With a steady internet connection, the system should load the main dashboard

in 3 seconds.

- **PER-2:** In less than two seconds following an update, the system will synchronize user data (tasks, attendance, and recitation) with Firebase.
- **PER-3:** Without the need for a manual refresh, the system will show task and attendance updates in real time.
- **PER-4:** Within five seconds of the appointed time, the user's device must receive notifications and reminders.
- **PER-5:** Up to 500 active students 50 teacher record and 200 tasks per account must be handled by the system while maintaining steady performance.

3.4.3 Reliability & Availability

While availability determines how easily a system may be accessed by users, reliability guarantees that the system will continue to operate as intended over time.

- **REL-1:** When the system is operating normally, excluding maintenance intervals, it must have an uptime of at least 99%.
- **REL-2:** In the event of a brief network outage, the system will instantly reconnect to Firebase, restoring synchronization without erasing data.
- **REL-3:** To guarantee recovery in the event of a data loss, all student, teacher, attendance and recitation data must be automatically backed up to the cloud once every 24 hours.
- **REL-4:** The system shall provide retry mechanisms for failed operations to ensure data integrity.
- **REL-5:** Upon restarting, the system will restore the most recent saved state in the event of an app crash or interruption.

3.4.4 Security

Security criteria specify how user information and system access are safeguarded. QMMS places a high priority on data confidentiality, secure authentication, and user privacy.

- **SEC-1:** Through a secure email-password login, the system will employ Firebase Authentication to confirm user identity.
- **SEC-2:** HTTPS/TLS protocols must be used to encrypt all client-server

communications.

- **SEC-3:** Passwords must never be sent in plain text and must always be saved in hashed form.
- **SEC-4:** To stop unwanted access, the system will use session timeout after ten minutes of inactivity.
- **SEC-5:** Role-based access control must be enforced to ensure users can only access features appropriate to their role (Admin, Teacher, Student).
- **SEC-6:** By blocking any unwanted access to analytics or personal data, the app will adhere to data protection regulations.

3.4.5 Scalability

Requirements for scalability specify how quickly the system can expand to accommodate higher user and data loads without seeing a decline in performance.

- **SCA-1:** The system architecture must be scalable to support multiple Madrassa (10,000 users at once) without compromising response times longer than three seconds.
- **SCA-2:** As the user base grows, the Firebase backend will enable dynamic expansion of bandwidth and data storage.
- **SCA-3:** New modules (such as parent portal or multiple Qira'at support) can be added to the program without requiring significant code reorganization thanks to its modular design.
- **SCA-4:** In the event that future improvements result in a 200% increase in data volume, the system will continue to operate steadily.
- **SCA-5:** To guarantee scalability and data integrity, the database structure must allow multi-user setups with separated storage for each institute.

Chapter 4

DESIGN AND ARCHITECTURE

4. Design and Architecture

QMMS is a cross platform mobile and desktop application designed to consolidate Madrassa management, Quran memorization tracking, attendance management, task assignment, AI-powered recitation mistake detection, performance analytics, and financial accounting into a single platform. It allows administrators to manage student and teacher accounts, assign teachers to students, and monitor overall Madrassa performance. Teachers can mark attendance across three shifts (Morning, Evening, Afternoon), assign specific Surah and Ayah ranges as memorization tasks, review AI-generated mistake reports, and track student progress. Students can view assigned tasks, record recitation using device microphone, receive AI-powered feedback, view highlighted mistakes, listen to correct recitation audio, track personal progress, and view attendance records. Additional modules include the Samaa AI Mistake Tracker for automated recitation analysis, performance reporting with visual charts, and accounting for donations and expenses management.

QMMS is implemented for Android, iOS and Windows platforms using Flutter for cross platform front end development, Firebase for cloud services including authentication, Firestore database, storage and real time synchronization. User authentication, role based access control and cloud synchronization ensure secure access and flexible control over Admin, Teacher and Student dashboards. The system emphasizes modularity, enabling scalability for future features like parent portal, multiple Qira'at support and advanced Tajweed error detection. By combining multiple functionalities in one app, QMMS minimizes the stress of juggling multiple manual systems while promoting memorization quality and institutional efficiency.

4.1 System Architecture

System architecture defines the overall structure of QMMS and explains how different components interact with each other. It provides a clear view of software modules, data flow, communication channels and technology layers used in the system. A well designed architecture ensures scalability, maintainability, performance and security. QMMS consists of a Flutter based cross platform frontend (mobile and desktop), cloud backend services, authentication modules, AI services and notification systems. The frontend handles user interaction, while the backend manages business logic, synchronization, analytics and data processing. Firebase is used for cloud data storage and real time synchronization. This layered architecture improves flexibility and allows future feature expansion.

4.1.1 Architectural Design

The system follows a Layered Architecture that separates the application into three main layers: the Presentation (UI) Layer, the Business Logic (Service) Layer, and the Data (Database) Layer. In Figure 4.1, the Presentation Layer includes three types of user's teacher, student and admin each interacting with the system through their respective desktops.

The UI Layer handles all user interaction through screens such as task assignment, attendance marking and student management, ensuring that presentation remains independent from logic. It forwards all functional requests to the Service Layer, keeping the interface simple and focused on displaying information.

The Application Layer contains the core logic, processing user actions and managing communication with the Data Layer. It handles all business rules, data validation, and service orchestration. Firebase Authentication handles secure sign in with role based access, while Firestore stores data such as student records, teacher assignments, attendance logs, tasks, recitation sessions, mistake reports, donations and expenses in structured collections. Firebase Storage manages audio files for recitation recordings and correct recitation playback. This modular separation ensures easier maintenance, scalability and clear coordination between layers. The layers interact in a clean flow where the user interface requests logic, logic processes and accesses data and results are returned to the user interface, creating a stable and extensible system architecture.

These application components process user requests and communicate with the Data Layer, which stores all necessary information, including attendance records, user details, assigned tasks (like Surah and Ayah ranges), progress data, Quranic content, mistakes, and recitation records. The Data Layer manages all data storage and retrieval operations. It uses Firebase Firestore for cloud database storage and Firebase Storage for audio file management.

At the bottom, the AI Processing Layer provides advanced features by using tools like an audio recorder, an automatic speech recognition (ASR) model, and string-matching algorithms to analyze recitations and highlight mistakes. Overall, the system is designed in a layered structure where each layer has a specific role, ensuring smooth interaction, efficient data management, and intelligent processing of Quran memorization activities.

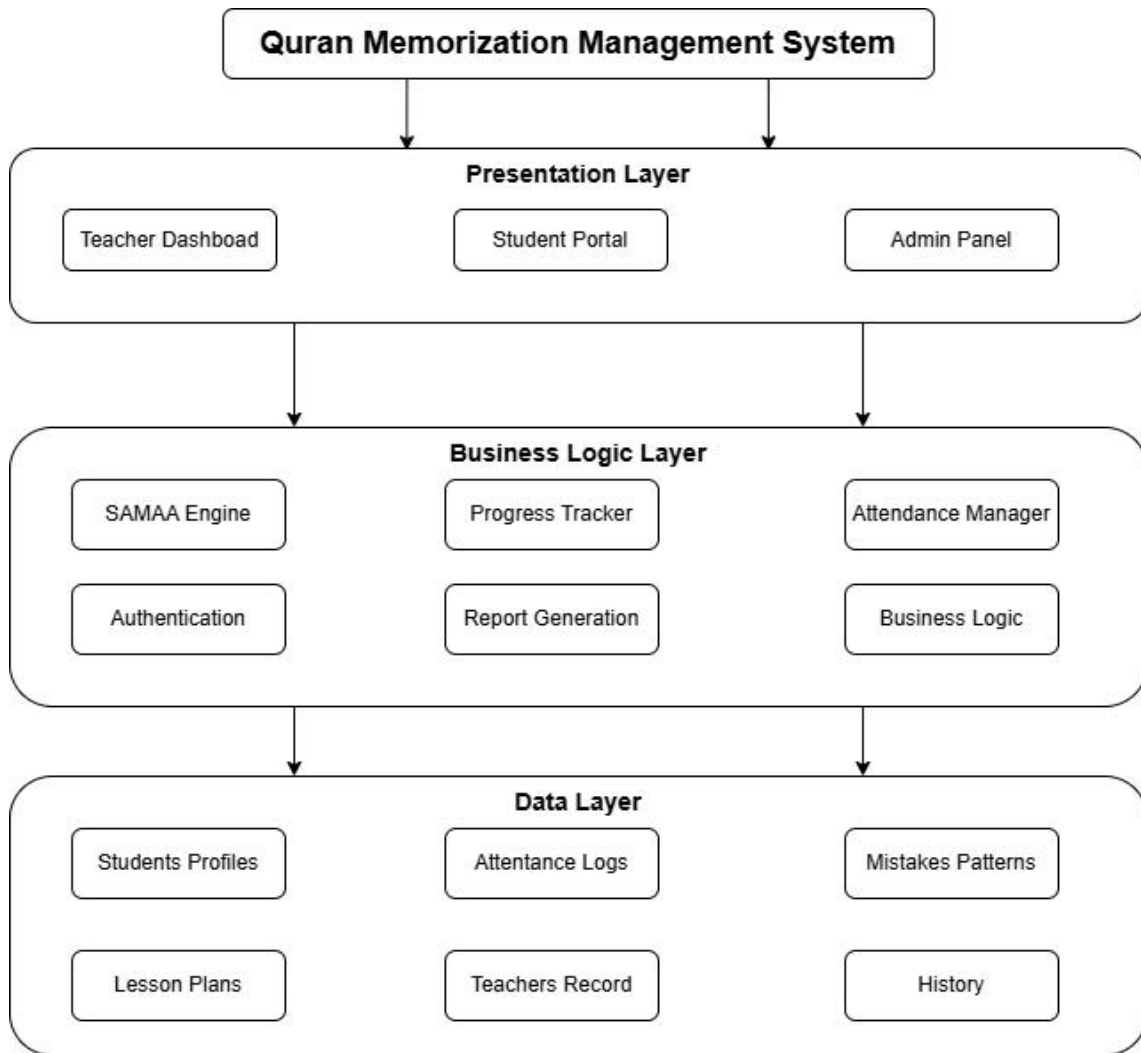


Figure 4.1: QMMS Architecture Diagram

4.2 Data Representation

In QMMS, the information domain is transformed into structured data entities that represent the core components of the system, such as users, students, teachers, attendance, tasks, recitations, mistakes, donations, and expenses. Each entity is modeled as an object with attributes (data fields) and methods (functions) that handle processing and updates.

Storage and Organization:

- **Firestore:** Used to store all user-related data, student records, teacher records, attendance logs, tasks, recitation sessions, mistake reports, donations, and expenses in real-time. This ensures that updates are synchronized across devices instantly.
- **Storage:** Used to store recitation audio files and correct recitation audio clips for playback.

- **Quran Database:** Stores Surah names, Ayah text in Arabic, and correct recitation audio clips referenced for comparison and playback.

Data Processing:

- Operations like creating tasks, marking attendance, recording recitation, detecting mistakes and updating progress are handled through the system's backend functions.
- The backend ensures data consistency, applies business rules (like Ayah range validation, mistake categorization), and updates both local and cloud storage as needed.

Major Entities:

- **User:** Stores account and role details (Admin, Teacher, Student).
- **Student:** Stores student profile, enrollment date, assigned teacher, and graduation status.
- **Teacher:** Stores teacher profile, joining date, and performance metrics.
- **Attendance:** Tracks student attendance with date, shift and status.
- **Task:** Stores Surah, Ayah range, assignment date, and completion status.
- **Recitation:** Tracks recitation sessions with audio file path, transcribed text, and submission date.
- **Mistake:** Stores mistake details including Ayah key, mistake type (Missed/Skipped/Repeated), and corrected text.
- **Donation:** Records donor name, amount, date, and payment mode.
- **Expense:** Records description, amount, date, and category.
- **Progress:** Tracks Juz completed, mistake trends, and weak areas.

This design ensures that all system data is organized, accessible and consistently updated, supporting both online functionality and real-time synchronization.

4.2.1 Data Dictionary

The Data Dictionary of the QMMS system defines the main data entities, attributes and data

types used in the QMMS system. It provides a structured description of how data is stored and managed within the application. The system uses Firebase Firestore for cloud storage. In Table 4.1, represents core modules of the application including users, students, teachers, attendance, tasks, recitations, mistakes, donations, expenses and progress tracking.

Table 4.1: QMMS Data Dictionary

Entity	Attribute Name	Data Type	Constraints	Description
Users	user_id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique identifier for each user (Firebase UID).
	email	VARCHAR(255)	UNIQUE, NOT NULL	Hashed and salted password.
	password_hash	VARCHAR(25)	NOT NULL	User role for access control.
	Profile_picture	TEXT	NULL	URL or path to profile image
	Created_at	TIMESTAMP	NOT NULL	Account creation timestamp
Students	student_id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique student identifier.
	user_id	VARCHAR(50)	FOREIGN KEY (Users.user_id)	Reference to user account
	Full_name	VARCHAR(25)	NOT NULL	Student's full name
	contact	VARCHAR(20)	NULL	Student's contact number
	enrollment_date	Date	NOT NULL	Date of enrollment.

	assigned_ teacher_id	VARCHAR(50)	FOREIGN KEY (Teacher.teacher_ id)	Assigned teacher reference
	graduation_ status	Boolean	DEFAULT FALSE	Whether student has graduated
Teachers	teacher_id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique teacher identifier.
	user_id	VARCHAR(50)	FOREIGN KEY (Users.usr id)	Reference to user account
	full_name	VARCHAR(255)	NOT NULL	Teacher's full name
	contact	VARCHAR(20)	NULL	Teacher's contact number
	joining_date	DATE	NOT NULL	Date of joining
	specialization	VARCHAR(100)	NULL	Teaching specialization
Attendance	attendance_ id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique attendance record ID
	student_id	VARCHAR(50)	FOREIGN KEY (Students. student_id)	Student reference
	teacher_id	VARCHAR(50)	FOREIGN KEY (Teachers.teacher_ id)	Teacher who marked attendance
	date	DATE	NOT NULL	Attendance date
	shift	ENUM ('Morning', 'Evening' , 'Afternoon')	NOT NULL	Attendance shif

	status	ENUM('Present', 'Absent')	NOT NULL	Attendance status
Tasks	task_id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique task identifier.
	teacher_id	VARCHAR(50)	FOREIGN KEY (Teachers.teacher_id)	Teacher who assigned task
	student_id	VARCHAR(50)	FOREIGN KEY (Students.student_id)	Student assigned task
	surah_no	INT	NOT NULL	Surah number (1-114)
	ayah_start	INT	NOT NULL	Starting Ayah number
	assigned_date	DATE	NOT NULL	Task assignment date
	status	ENUM('Pending', 'In-Progress', 'Completed')	DEFAULT 'Pending'	Task completion status
Recitations	recitation_id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique recitation session ID
	student_id	VARCHAR(50)	FOREIGN KEY (Students.student_id)	Student who recorded
	task_id	VARCHAR(50)	FOREIGN KEY (Tasks.task_id)	Associated task
	audio_file_path	TEXT	NOT NULL	Path to recorded audio
	Transcribed_text	TEXT	NULL	ASR transcribed text

	Submission_date	TIMESTAMP	NOT NULL	Submission timestamp
	total_mistakes	INT	DEFAULT 0	Total mistakes detected
Mistakes	mistake_id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique mistake identifier
	recitation_id	VARCHAR(50)	FOREIGN KEY (Recitations.recitation_id)	Associated recitation
	ayah_key	VARCHAR(50)	NOT NULL	Ayah identifier
	mistake_type	ENUM('Missed', 'Skipped', 'Repeated')	NOT NULL	Type of mistake
	correct_text	TEXT	NOT NULL	Correct Quranic text
Donation	donation_id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique donation identifier
	donor_name	VARCHAR(255)	NOT NULL	Name of donor
	amount	DECIMAL (10,2)	NOT NULL	Donation amount
	donation_date	DATE	NOT NULL	Date of donation
	Payment_mode	VARCHAR(50)	NULL	Payment method
Expenses	expense_id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique expense identifier

	description	TEXT	NOT NULL	Expense description
	amount	DECIMAL(10,2)	NOT NULL	Expense amount
	expense_date	DATE	NOT NULL	Date of expense
	category	VARCHAR(100)	NULL	Expense category
Progress	progress_id	VARCHAR(50)	PRIMARY KEY, NOT NULL	Unique progress record ID
	student_id	VARCHAR(50)	FOREIGN KEY (Students.student_id)	Student reference
	total_juz_completed	INT	DEFAULT 0	Number of Juz memorized
	total_mistakes	INT	DEFAULT 0	Total mistakes recorded
	weak_areas	JSON	NULL	Surahs/Ayat with most mistakes
	last_activity_date	DATE	NULL	Last recitation date

The ER diagram of QMMS illustrates the structural relationships among the main entities of the system, including Users, Students, Teachers, Attendance, Tasks, Recitations, Mistakes, Donations, Expenses and Progress.

In Figure 4.2, provides a clear and organized view of how data is stored, managed and interconnected across different modules of the application. Each entity represents a core functional component of the system, ensuring that all Madrassa activities are systematically

recorded and maintained in a structured database design.

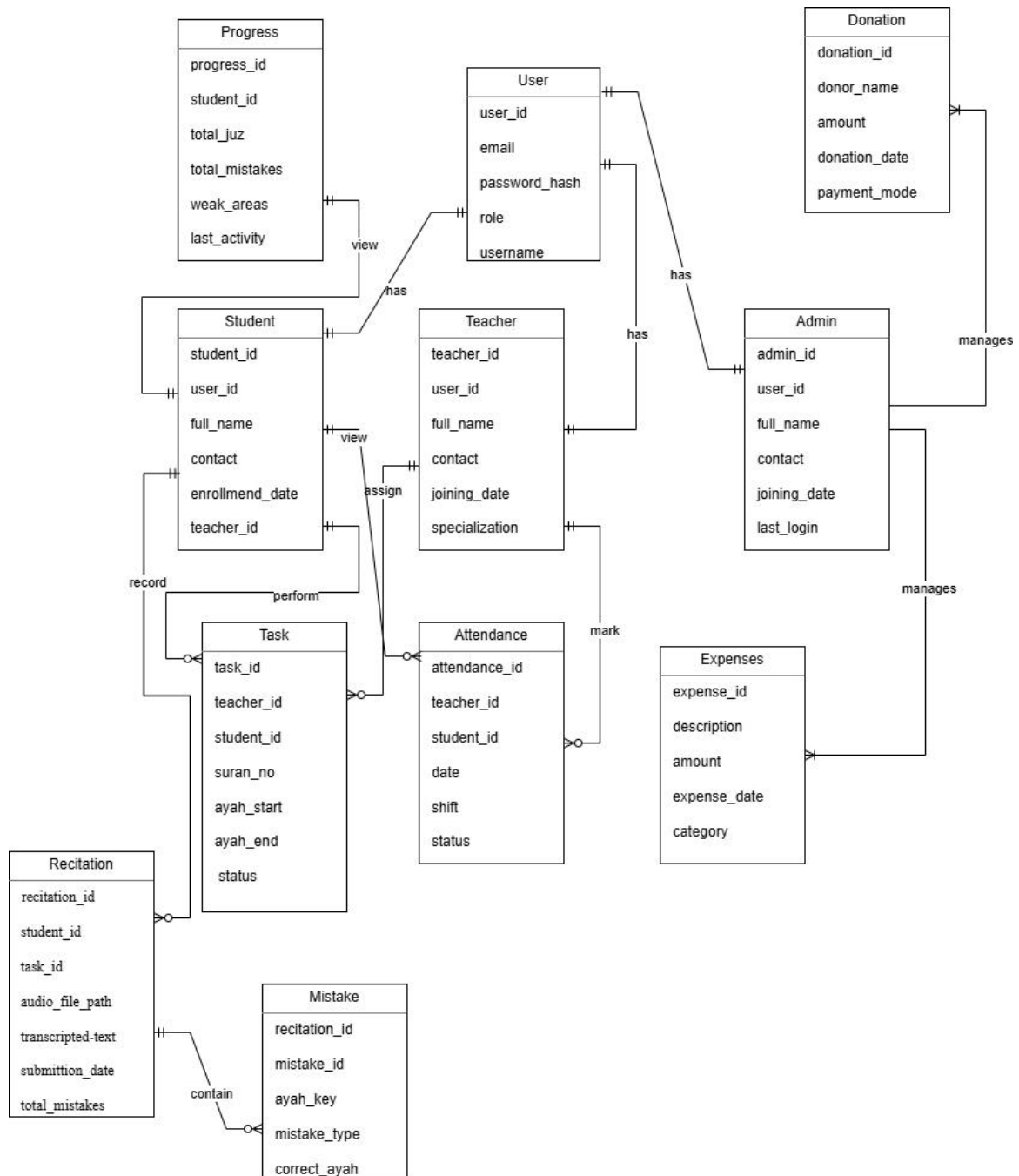


Figure 4.2: QMMS Entity Relationship Diagram

The User entity serves as the central entity of the system, where each user is linked to role-specific entities such as Students and Teachers. This establishes a one-to-one or one-to-many relationship between Users and these roles.

The Teacher entity serves as the instructional backbone of the system, connected to Students, Tasks and Attendance for all their assigned students.

The student entity is central to the system, connected to Attendance, Tasks, Recitations, and

Progress. The Teacher entity is connected to Students, Tasks, and Attendance. The Recitation entity is connected to Mistakes and each Mistake is categorized by type (Missed, Skipped, or Repeated Ayah). The Donation and Expense entities are independent but linked through the admin user for financial reporting.

Overall, the ER diagram ensures proper relationship mapping with clear primary and foreign key constraints, data consistency through referential integrity, and efficient organization of the QMMS system across all functional modules. The logical separation of entities based on business domains (authentication, academics, attendance, financial, progress) supports maintainability, scalability, and future feature expansion while ensuring smooth functionality across all modules.

This structured database design forms the foundation of the QMMS system, enabling reliable data storage, retrieval, and analysis for effective Madrasa management and Quran memorization tracking.

4.3 Process Flow Representation

This section describes the major processes within the QMMS system and their sequence of operations. The focus is on key workflows critical to the platform's functionality.

User Registration and Login

- Users access the registration or login page.
- New users provide their personal details and create a secure account. Existing users enter their credentials to log in.
- The system verifies the credentials using Firebase Authentication and redirects users to the role specific dashboard upon successful authentication.

Admin Workflow

- Admin assign students to the teachers from the list of unassigned students.
- Admin navigates to the progress dashboard, views overall student progress.
- Admin navigates to the financial module, records donations with donor name, amount, date, and payment mode, records expenses with description, amount, date, and category, and views financial summaries.

Teacher Workflow

- Teacher navigates to attendance module and mark student's attendance.

- Teacher navigates to the task assignment module, selects a student, selects Surah from dropdown list, enters Ayah start and end, and assigns the task.
- Teacher navigates to mistakes module and review students mistakes.

Student Workflow

- Student navigates to the tasks module, views list of assigned tasks.
- Student selects a task, clicks "Start Recitation," allows microphone permissions, clicks "Start Recording," recites assigned verses, clicks "Stop Recording," previews recording, and clicks "Submit for Analysis." The system uploads audio for SAMAA AI processing.
- Student navigates to the mistake report module and view mistakes.
- Student navigates to the progress dashboard, views memorized vs remaining Juz.

Sameea AI Processing

- Audio file is uploaded to Firebase Storage.
- Cloud ASR service converts speech to Arabic text.
- System fetches correct Quranic text from Quran Database.
- System normalizes both texts (removes diacritics).
- System compares texts using Levenshtein distance string matching algorithm.
- System detects missed Ayah, skipped Ayah, repeated Ayah.
- System highlights mistakes in Quranic text
- System generates detailed mistake report.
- System stores session and updates student progress.

4.4 Design Model

A Design Model is a blueprint of a software system that describes how the system will be structured and implemented. It transforms the requirements gathered during the analysis phase into a detailed technical design that developers use to build the software. The design model defines the architecture, components, interfaces, database structure, and interactions between different modules of the system.

In simple words, the design model explains how the software will be developed to meet the specified requirements.

4.4.1 Class Diagram

A Class Diagram is a static UML (Unified Modeling Language) diagram that represents the structure of a software system. It shows the classes, their attributes (data), methods (functions), and the relationships between different classes. A class diagram acts as a blueprint for developers by illustrating how the objects in the system are organized and how they interact with one another

The User is the central class, with Admin, Teacher, and Student inheriting from User.

The Student class owns Attendance, Task, Recitation, and Progress, ensuring nothing can exist without a logged-in user. In Figure 4.3, the Student class includes attributes such as `student_id`, `full_name`, `contact`, `enrollment_date`, `graduation_status`, and a foreign key reference to Teacher (`teacher_id`). Student methods include `viewTasks()`, `recordRecitation()`, `viewMistakes()`, `viewProgress()`, and `viewAttendance()`. The class ensures that all academic activities are properly linked to the authenticated student user. The

Teacher class is associated with Student and Task. The Recitation class is associated with Mistake. The Teacher class manages multiple students and assigns multiple tasks. The Sameea AI class encapsulates the core AI functionality with methods for speech-to-text conversion, text comparison, mistake detection, and report generation, ensuring modularity and separation of concerns.

Mistake has an enum for `mistake_type`. The class diagram also includes Donation and Expense classes linked to Admin for financial management, and Mistake class linked to Recitation for tracking.

The Recitation class is associated with Mistake. The Recitation class captures each student's recitation session with attributes including `recitation_id`, `student_id` (foreign key), `task_id` (foreign key), `audio_file_path`, `transcribed_text`, `submission_date`, and `total_mistakes`. Methods include `save()`, `submit()`, `getByStudent()`, `getByTask()`, `getMistakes()`, and `getByDateRange()`. This modular design allows future enhancements such as Tajweed rule grading or multiple Qira'at support to be added with minimal impact on existing code.

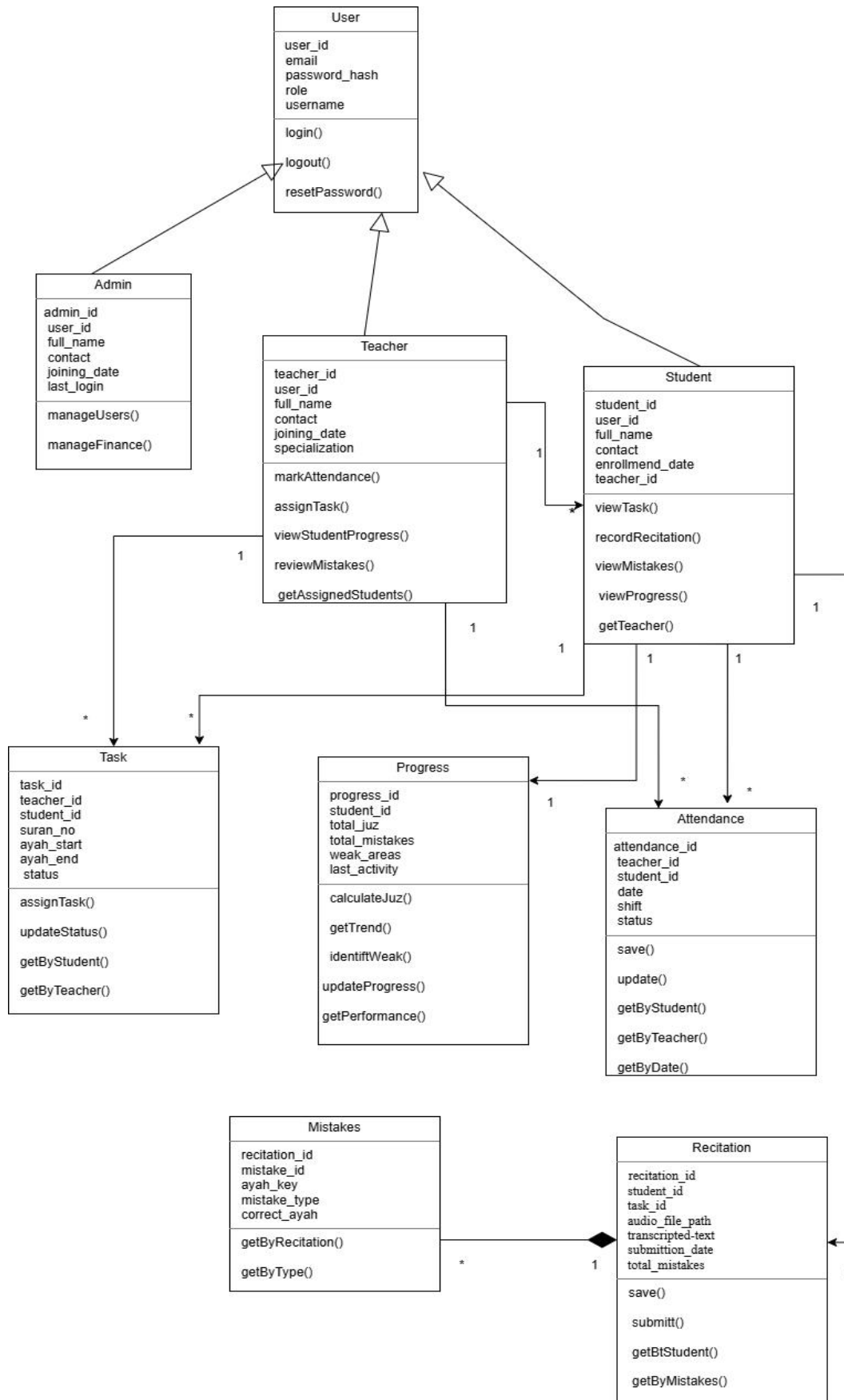


Figure 4.3: QMMS Class Diagram

4.4.2 Sequence Diagram

The sequence diagram outlines user interaction with the desktop application and its components. It begins with user login and role verification, followed by actions like recitation recording, mistake detection, attendance marking, or report viewing.

Performance Analysis Sequence Diagram

Figure 4.4 shows sequence diagram of how a student interacts with the system to complete a recitation task and review performance.

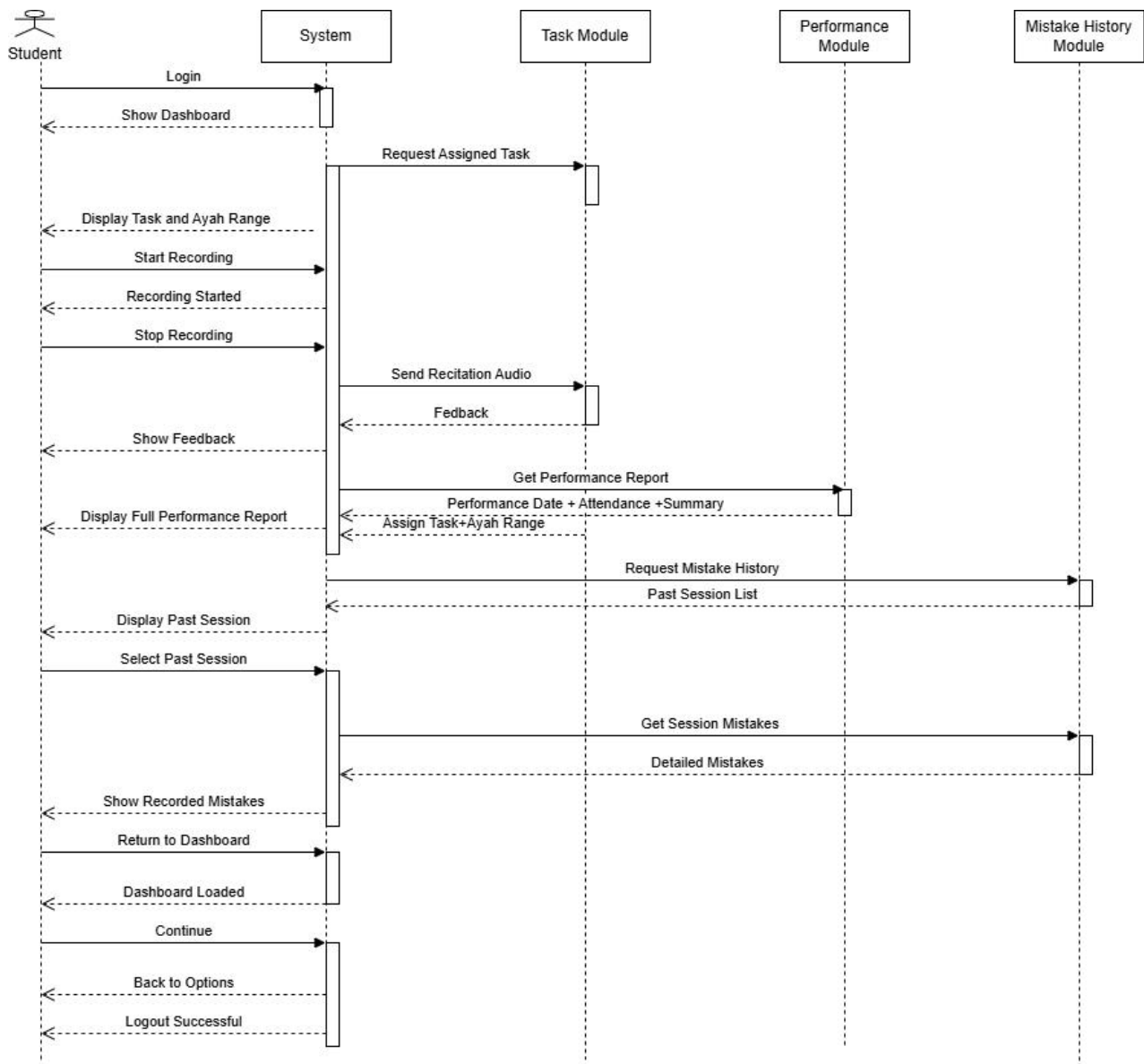


Figure 4.4: Performance Analysis Sequence Diagram

Sequence Diagram of Mistakes Detection

Figure 4.5 shows Samaa Module is the core AI-powered component of the Quran Memorization Management System that automatically analyzes student recitation and detects mistakes. This diagramme shows the sequence of Samaa AI module.

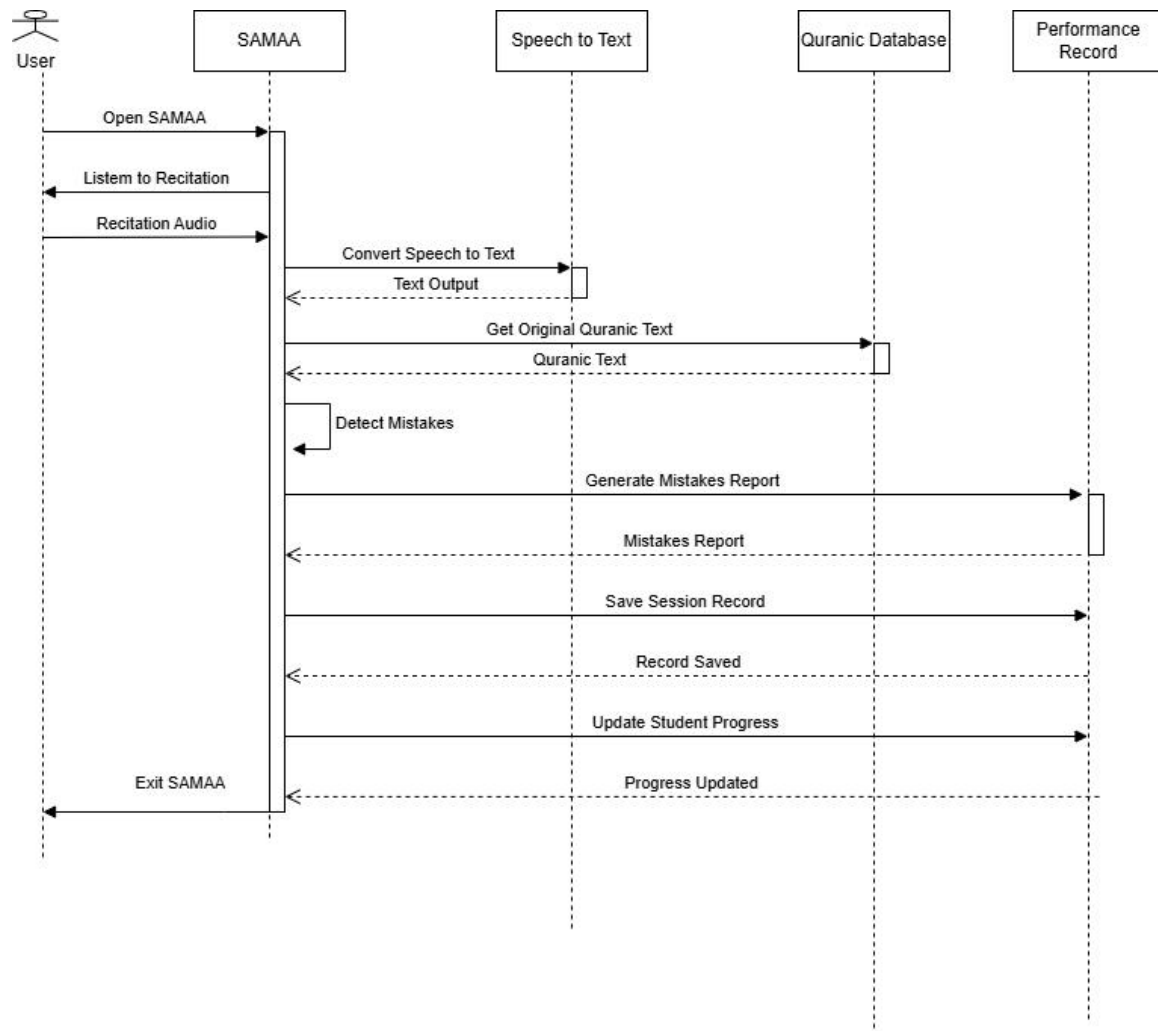


Figure 4.5: Mistakes Detection Sequence Diagram

Sequence Diagram of Student Management

Figure 4.6 describes how teacher login into the system and manage students related activities. The teacher can mark attendance, assign recitation task and view or update student performance.

After completing tasks, the system returns to the dashboard. his process begins with the teacher signing up or logging in and accessing the dashboard, after which they can mark attendance by opening the attendance module the system records and saves this data in the database. The teacher then selects and assigns tasks through the task module, and the

assignment is stored successfully. Next, the teacher accesses the performance module to check and update student performance and progress, with all changes saved in the database.

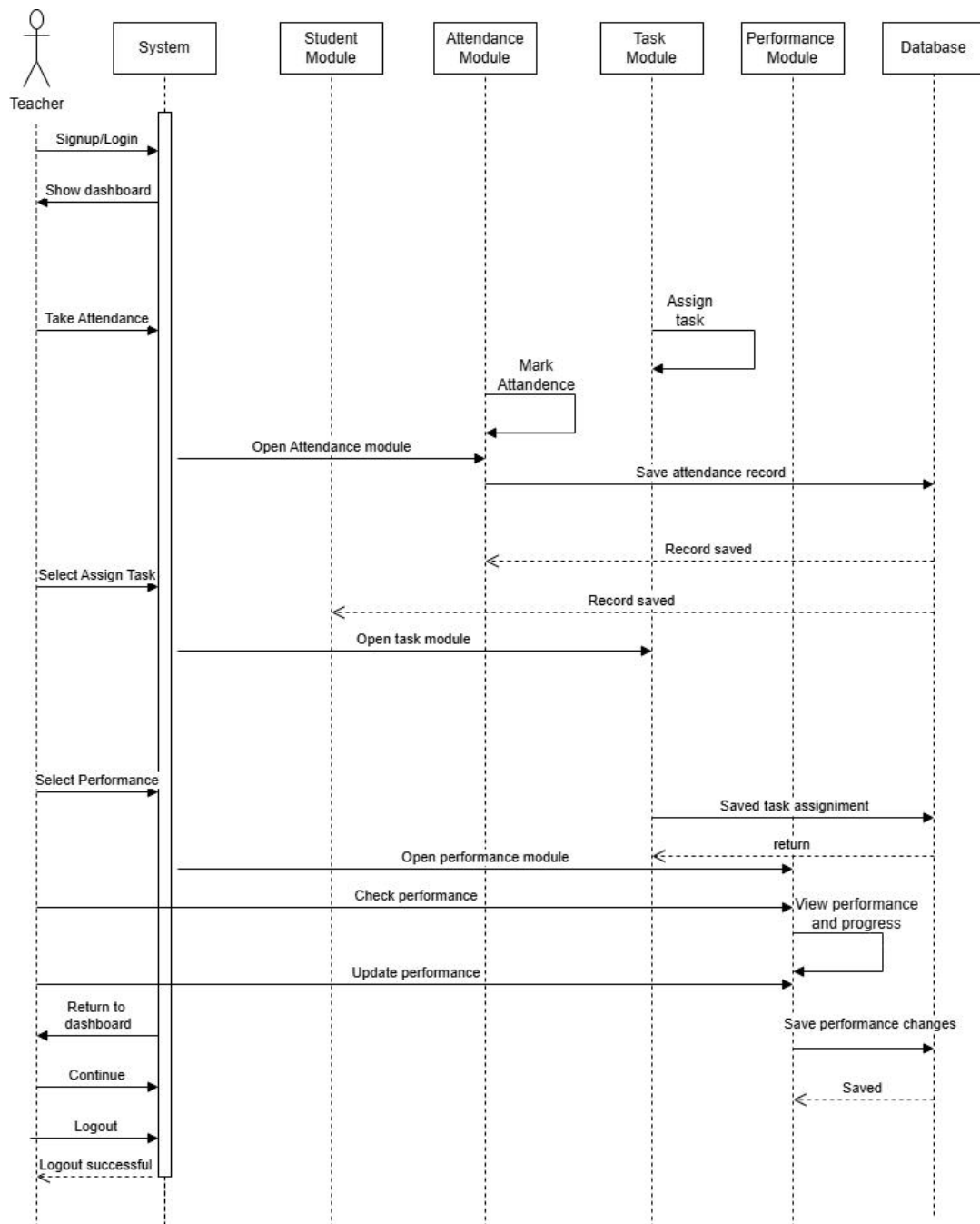


Figure 4.6: Student Management Sequence Diagram

Sequence Diagram of Manage Accounts

Admin login into the system and manage related activities. The user then clicks on “View Progress,” and the system loads the progress page. When the user requests to view the “Students/Teacher” list, the web server retrieves the list data from the database and displays it.

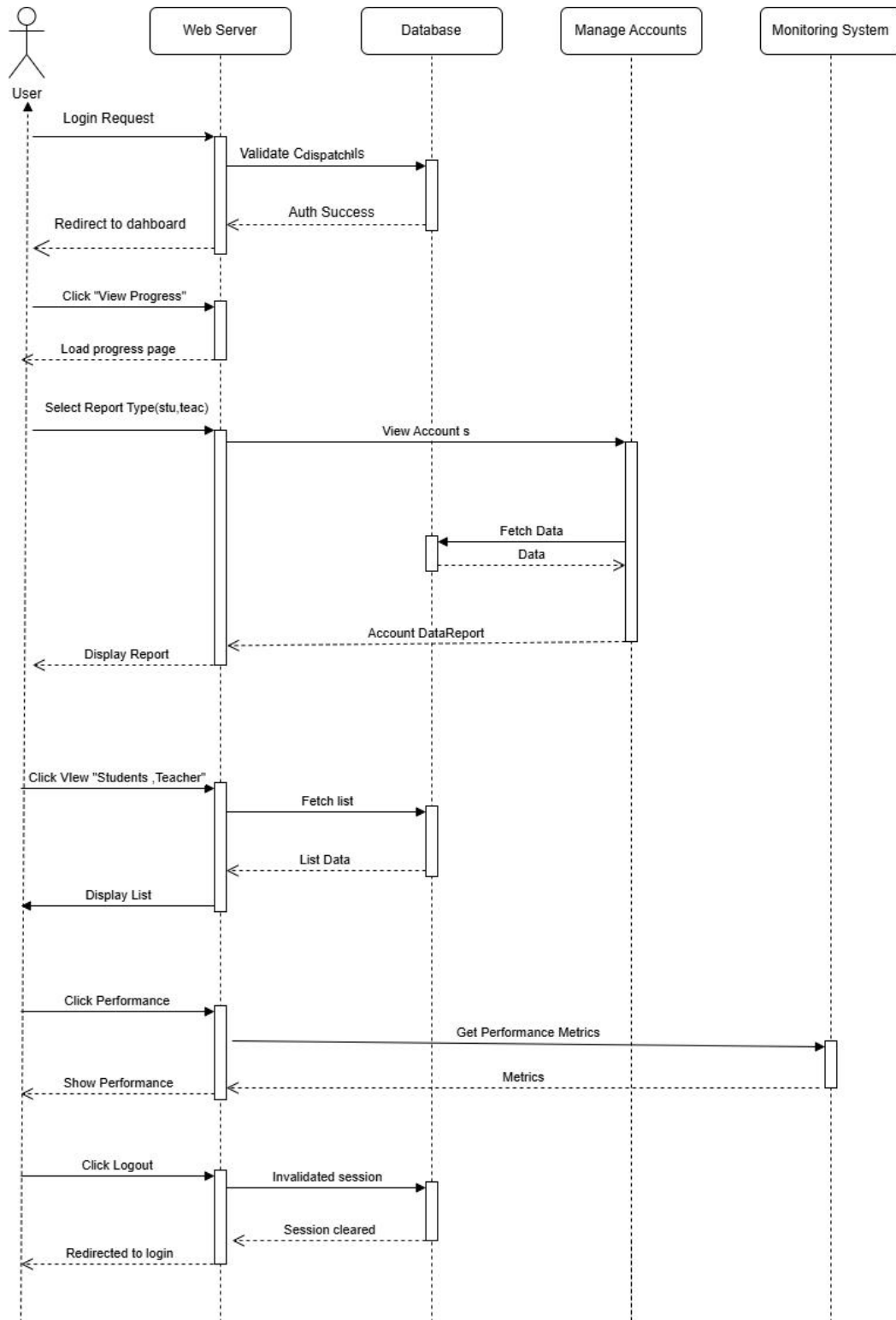


Figure 4.7: Manage Accounts(teacher, student profile)

In Figure 4.7 the admin can manage donations and finances, assign assign students and view teacher and student performance.

4.4.3 Process Flow Diagram

The system's process flow begins with user authentication, where individual's login to the desktop application and are directed to their role-specific dashboard (Admin, Teacher, or Student). Once logged in, students can view assigned tasks, record recitation using a microphone, submit to the AI Mistake Tracker, view highlighted mistakes with correct recitation playback, and track personal progress. Teachers can mark attendance, assign Hifz tasks including specific Surah and Ayah ranges. Admins can manage teacher and student accounts, assign teachers to students, monitor overall student progress, evaluate teacher performance and view attendance summaries. In the background, the AI Mistake Tracker continuously processes recitation submissions,.

QMMS Flow Diagram

Figure 4.8, shows the complete user workflow of the Quran Memorization Management System (QMMS), showcasing how different user roles interact with the system's core functionalities. The flow begins when a user opens the application and logs in, after which they select their designated role from three options: Admin, Teacher, or Student. Based on the selected role, the system redirects the user to their respective role-specific dashboard with appropriate permissions and features.

The Admin workflow focuses on institutional oversight, enabling the administrator to view teacher performance metrics including graduation rates and average completion times, monitor student performance across all teachers, access comprehensive Madrassa statistics such as donation and expenses maintenance.

The Teacher workflow centers on day-to-day instructional management, allowing teachers to mark attendance across shifts, assign memorization tasks by selecting specific Surah and Ayah ranges for individual students, review AI-generated mistake reports , track student performance over time including Juz completion and mistake trends.

The Student workflow is the most detailed, beginning with selecting an assigned task from their teacher, then starting recitation recording using the device microphone with real-time feedback, reciting the assigned verses, and stopping the recording for submission.

The Samaa AI module serves as the core intelligent component bridging the gap between student practice and teacher feedback while automating the mistake detection process that

traditionally required manual teacher intervention, providing instant feedback to students and enabling data-driven institutional management.

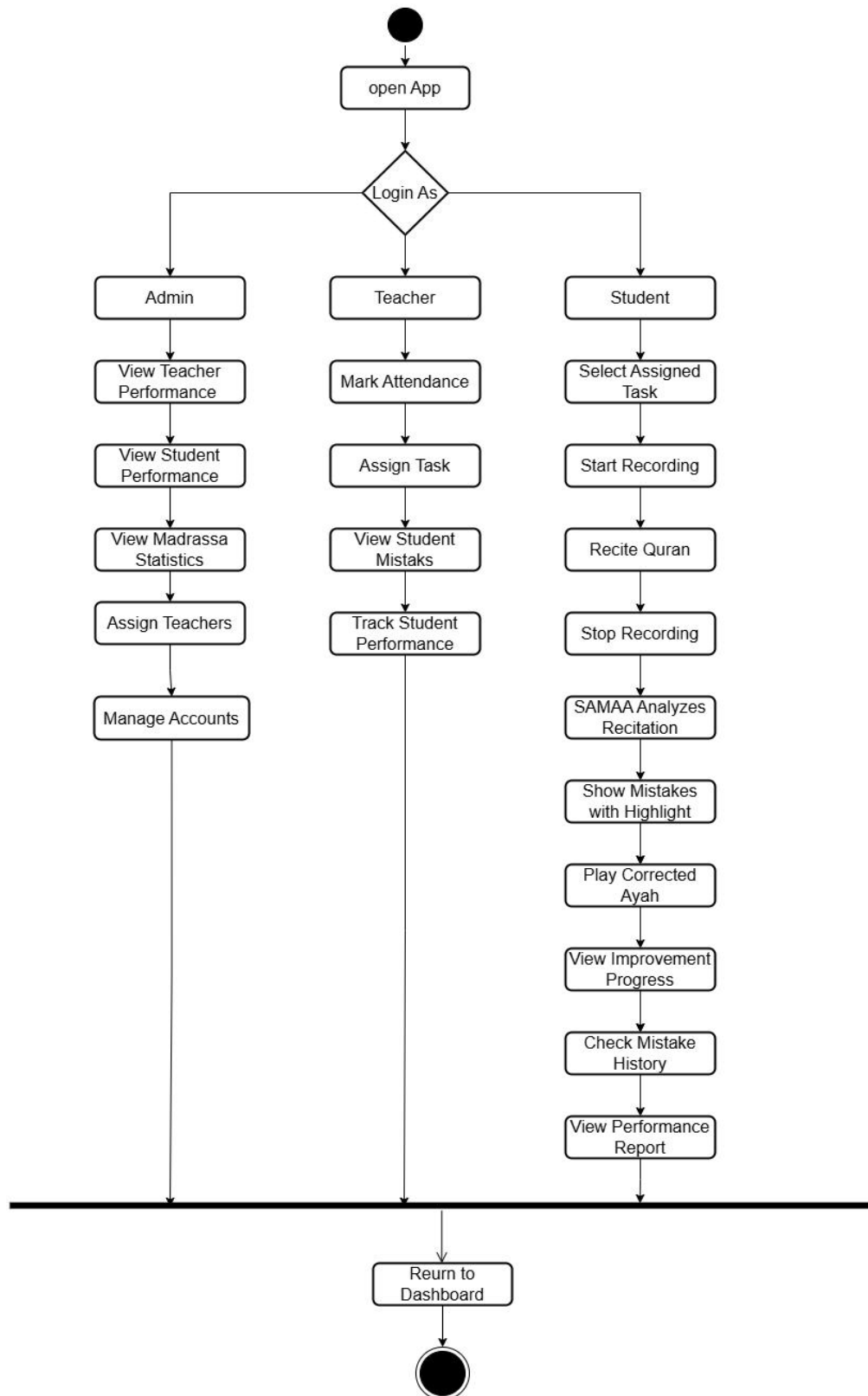


Figure 4.8: QMMS Flow Diagram

Samaa AI Flow Diagram

Figure 4.9 shows the workflow of the Sameea module in the Quran Memorization Management System. The Sameea module records the student's recitation, converts the speech into text, and compares it with the corresponding Quranic text. It detects recitation mistakes, highlights the incorrect Ayah, and plays the correct Ayah to support learning.

30

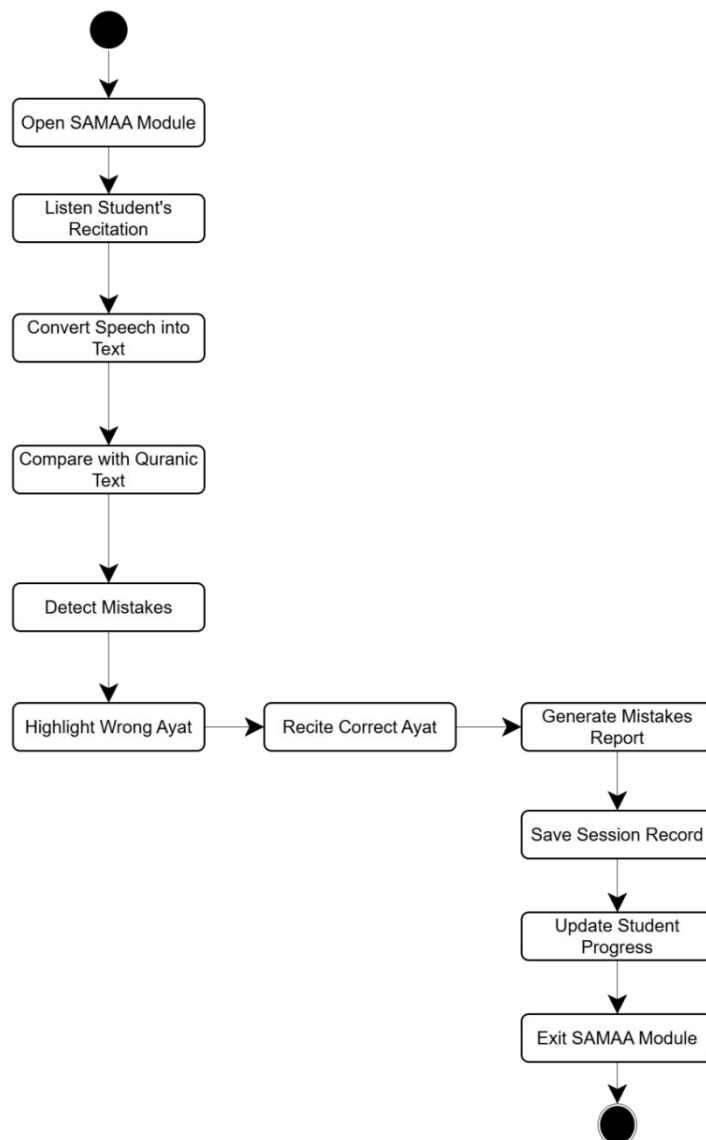


Figure 4.9: Samaa AI Flow Diagram

The module then generates a mistake report, stores the recitation session, updates the student's progress, and finally exits the Sameea module after saving all the required data.

Chapter 5

IMPLEMENTATION

5. Implementation

This chapter outlines the implementation details of the Quran Memorization Management System project, describing the algorithms used, system modules developed, database structure, and the core functionalities integrated into the Windows desktop application. It also references UML diagrams that visually represent the system's architecture, workflow and interaction between components.

The implementation of QMMS is the result of systematic planning , detailed design and the integration of intelligent features such as an AI powered Mistake Tracker SAMAA module, and a comprehensive performance reporting system. This chapter explains how the system was practically developed including the logic behind the AI Mistake Tracker, the data flow for attendance and task management, the admin management features, and the interaction design used to create an intuitive user experience.

By covering algorithms, data processing, and UI development, this chapter bridges the gap between initial system architecture and the final functional desktop application. Each section provides a clear explanation of the purpose, scope, and contribution of the individual components to the working of the overall Quran Memorization Management System.

5.1 Algorithm

The system's behaviour and logic are driven by core algorithms designed to ensure accuracy, smooth recitation analysis, and efficient access to memorization data. These algorithms define how the desktop application processes user input, retrieves relevant data, and displays results across key modules such as AI Mistake Tracker, attendance management, task assignment, and performance reporting. The core functionalities of Quran Memorization Management System are supported by several key algorithms that manage system operations. Below is the pseudocode-style or natural language explanation of how major system features work

5.1.1 User Authentication: This algorithm defines the secure process for user registration, login and password recovery in the QMMS system to ensure authenticated access to the application.

Steps:

1. Open the QMMS app and tap "Sign Up" or "Login."
2. Select authentication method: email/password or social login.

3. Enter required credentials (email, password) and submit.
4. If credentials are invalid → view error message and retry.
5. If password is forgotten → tap "Forgot Password" → enter email → reset via OTP or link.
6. On successful validation → get redirected to the home dashboard.
7. If authentication service is unavailable → wait for system retry notification.

5.1.2 Student Profile Management: Explains how administrators and teachers manage student profiles, including registration, assignment to teachers, and profile updates.

Steps:

1. Log with admin role or teacher role.
2. Navigate to the Students section.
3. Tap "Add Student" to register a new student.
4. Enter student details: full name, contact information, enrollment date.
5. Select an assigned teacher from the dropdown list.
6. Tap "Save" → student profile is created and linked to the teacher.
7. To edit student profile → select student → update fields → save changes.
8. To view student progress → select student → view dashboard with recitation history and mistakes.
9. If save fails due to network error → tap "Retry" to save again.
10. View all active students organized by teacher assignment.

5.1.3 Teacher Management: This algorithm describes how administrators manage teacher records, including adding, updating, and assigning teachers to students.

Steps:

1. Log in with admin role.
2. Navigate to the Teachers section.
3. Tap "Add Teacher" to register a new teacher.
4. Enter teacher details: full name, contact information, joining date, and specialization.
5. Tap "Save" → teacher profile is created.

6. To edit teacher profile → select teacher → update fields → save changes.
7. To delete a teacher → confirm removal (students must be reassigned first).
8. View teacher's assigned students and their progress.
9. If teacher is assigned to students, reassign students before deletion.
10. Generate teacher performance reports based on student progress.

5.1.4 Attendance Tracking: This describes how teachers mark and manage student attendance for different shifts and sessions.

Steps:

1. Log in as a teacher.
2. Navigate to the Attendance section.
3. Select the date for attendance marking (default: current date).
4. Select shift .
5. System displays list of students assigned to the teacher.
6. Mark attendance for each student: Present, Absent, or Late.
7. Tap "Save Attendance" → attendance records are saved to the database.
8. To view attendance history → select student → view attendance log.
9. If attendance save fails → system retries automatically.
10. View attendance summary and reports for each student.

5.1.5 Lesson Plan Assignment: This describes how teachers create and assign lesson plans (tasks) to students for Quran memorization.

Steps:

1. Log in as a teacher.
2. Navigate to Lesson Plans section.
3. Tap "Create New Lesson" to assign a new task.
4. Select student(s) from the dropdown list.
5. Enter Surah number and Ayah start/end verses.
6. Enter assignment description and deadline.

7. Tap "Assign" → lesson plan is created and linked to students.
8. Students can view assigned lessons in their dashboard.
9. Track completion status of each assigned lesson.
10. If lesson creation fails → retry or contact administrator.

5.1.6 Recitation Recording and Submission: This describes the process of students recording and submitting their recitation for evaluation.

Steps:

1. Log in as a student.
2. Navigate to assigned tasks/lessons.
3. Select a lesson to complete.
4. Tap "Start Recitation" to begin recording.
5. System records audio using device microphone.
6. Tap "Stop" when recitation is complete.
7. System automatically transcribes the recitation.
8. Tap "Submit" → recitation is saved with audio path and transcription.
9. System notifies the teacher for evaluation.
10. View submission history and teacher feedback.

5.1.7 Mistake Pattern Analysis: This describes how the system analyzes mistake patterns to identify weak areas and generate progress reports.

Steps:

1. System collects all mistake records for a student.
2. Group mistakes by type and Ayah references.
3. Identify recurring mistakes (pattern detection).
4. Calculate mistake frequency by Surah
5. Generate weak areas report for the student.
6. Update student progress metrics.
7. Teacher can view comprehensive mistake analysis.
8. Generate personalized recommendations for improvement.

9. System tracks improvement over time
10. Update progress record with last activity date.

5.1.8 Financial Management-Donation: This describes how the system records and manages donations received by the organization.

Steps:

1. Log in with admin role.
2. Navigate to Donations section.
3. Tap "Record Donation" to add a new donation.
4. Enter donor name, amount, and donation date.
5. Select payment mode: Cash, Bank Transfer, Online Payment, etc.
6. Tap "Save" → donation record is saved to the database.
7. View all donations with total amount summary.
8. Search and filter donations by date range or donor name.
9. Generate donation reports for financial review.
10. Export donation data for accounting purposes.

5.1.9 Financial Management-Expenses: This describes how the system records and manages organizational expenses.

Steps:

1. Log in with admin role.
2. Navigate to Expenses section.
3. Tap "Record Expense" to add a new expense.
4. Enter description of the expense.
5. Enter amount and expense date.
6. Select category: Salaries, Utilities, Maintenance, etc.
7. Tap "Save" → expense record is saved to the database.
8. View all expenses with total amount summary.
9. Search and filter expenses by category or date range.
10. Generate expense reports and budget analysis.

5.2 External APIs

External APIs (Application Programming Interfaces) are services provided by third-party platforms that allow applications to access specific functionalities without building them from scratch. They act as bridges between the QMMS application and external cloud services, enabling features like speech recognition, authentication, database management, file storage, and push notifications. APIs define a set of rules and protocols for how different software components should interact, allowing QMMS to leverage powerful cloud-based services while focusing on its core Madrasa management and Quran memorization features.

5.2.1 Firebase Authentication API: This API handles all user identity and security features within the QMMS. It powers the Login, Signup and Reset Password screens by managing secure account creation, encrypted credential verification and automated password recovery emails. Within the Profile and Settings modules, it ensures that only authorized users can access and modify personal data, while providing persistent session management so users stay logged in across app restarts. Role based access control is implemented to differentiate between Admin, Teacher and Student permissions.

5.2.2 Cloud Firestore API: Firestore acts as the primary real time database API, facilitating the storage and synchronization of all user generated content. In the Student and Teacher Management modules, it handles the creation, updating and deletion of profiles with instant cloud sync. The Attendance Logs, Lesson Plans and Recitation History modules rely on it to save detailed records and transcripts, while the Progress module uses its real time listeners to power progress dashboards and student monitoring. Financial data (Donations and Expenses) is also stored securely in Firestore.

5.2.3 SQL Database API(MySQL/PostgreSQL): The SQL Database API handles complex reporting and analytical queries for the QMMS. It is used for generating comprehensive reports on attendance, student progress, teacher performance and financial summaries. The API supports advanced querying capabilities for multi table joins, aggregation functions (SUM, COUNT, AVG) and date range filtering. It is integrated with Firestore for hybrid data management, where Firestore handles real time operational data and SQL handles analytical and historical reporting.

5.2.4 Audio Recording API: This local device API is utilized in the Recitation Submission module to capture student recitations. The API provides interfaces to access the device microphone, record high quality audio and save audio files to device storage. It supports

audio playback for teacher evaluation. Audio files are stored locally and uploaded to cloud storage for backup and teacher access. The API ensures audio quality suitable.

5.2.5 File Storage API (Cloud storage): This API is used to securely store and retrieve large files, including recitation audio files and user avatars. It provides scalable, durable storage with CDN delivery for fast access. Audio files are stored with appropriate metadata (student ID, date, Surah reference) for easy retrieval during teacher evaluations. Storage access is secured with Firebase Authentication rules to ensure only authorized users can access specific files.

5.2.6 Shared Preferences API: This is a lightweight data persistence API used primarily in the Settings and App Layout modules to store local user preferences. It tracks the user's choice of theme and caches basic profile information like the user's name and role. By storing these small key value pairs locally on the device, the app can immediately apply the user's preferred visual theme and display their name on the Home Page before the Firestore network calls complete. It also caches user role information for quick authorization checks.

5.3 User Interface

This presents the graphical user interface (GUI) of the QMMS application. It includes the main screens of the system along with their descriptions and functionalities. Each screen is designed to provide a simple and user friendly for users while interacting with different modules such as student management, attendance, lesson planning, recitation evaluation and financial management. The interface ensures smooth navigation and consistency across all features of the application.

The GUI of QMMS follows a role-based design approach, where the interface adapts dynamically based on the logged-in user's role. Administrators are presented with comprehensive management dashboards showing institutional analytics, student enrollment statistics, teacher performance metrics, and financial summaries. Teachers access dedicated interfaces for marking attendance across three shifts, assigning memorization tasks with Surah and Ayah selection, and reviewing AI-generated mistake reports with color-coded highlighting. Students interact with a personalized learning portal featuring their assigned tasks, recording controls for recitation practice, interactive mistake reports with correct audio playback, and visual progress charts showing Juz completion and mistake trends. This role-based segregation ensures that each user sees only relevant features, reducing cognitive load and improving productivity.

- **Welcome Screen:** The Figure 5.1 illustrates the welcome screen of the Quran Memorization Management System (QMMS). The screen prominently displays the title "القرآن الكريم" (The Holy Quran) in Arabic, followed by the English title "Quran Memorization Management System," establishing the application's identity and purpose. A Quran icon is positioned at the center to visually represent the system's focus on Quran memorization and learning. The interface adopts a soft beige and brown color scheme inspired by Islamic aesthetics, providing a clean, attractive, and user-friendly design.



Figure 5.1: Welcome Screen

This welcome screen appears when the application is launched, initializes the required resources, and then automatically navigates users to the login screen for secure authentication.

- **SignIn Screen:** In Figure 5.2, The Login Screen of the Quran Memorization Management System features a warm and inviting interface with the greeting message "Welcome Back" displayed prominently at the top, followed by the subtitle "Sign in to continue your journey" which reinforces the spiritual and educational nature of the application..

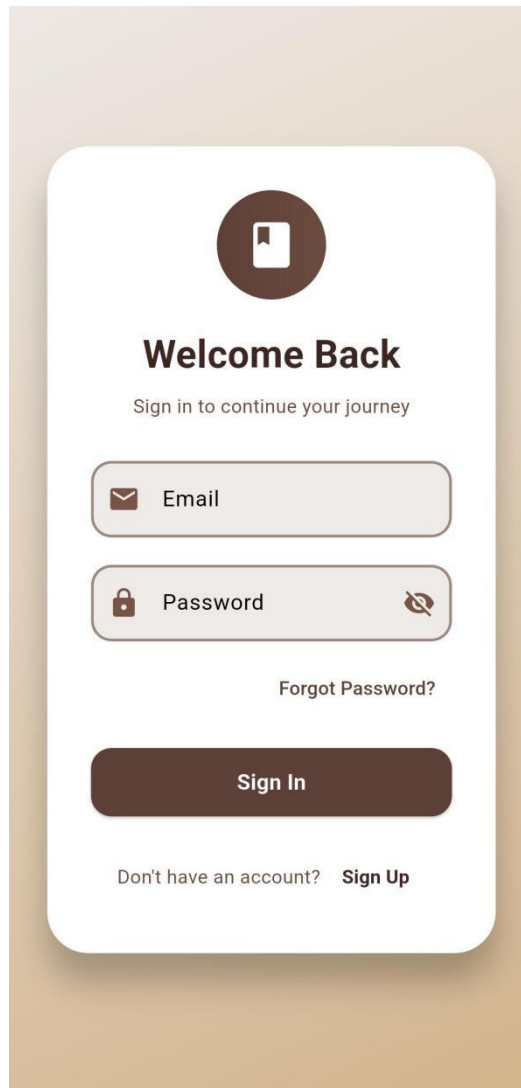


Figure 5.2: SignIn Screen

Below this welcome message, the screen presents two standard input fields: an Email field where users enter their registered email address, and a Password field where users securely enter their password with hidden characters for privacy protection. A "Forgot Password?" link is provided below the password field, allowing users to recover or reset their password if needed.

- **Student Dashboard Screen:** The Student Dashboard Screen opens with a personalized greeting "Good Morning ali" and the inspiring Quranic verse "وَقُلْ رَبِّ زِدْنِي عِلْمًا" (My Lord, increase me in knowledge — Surah Taha 20:114). The "CONTINUE LAST READ" section displays "Surah Al-Baqarah - Ayat 255" allowing students to resume memorization from where they stopped..

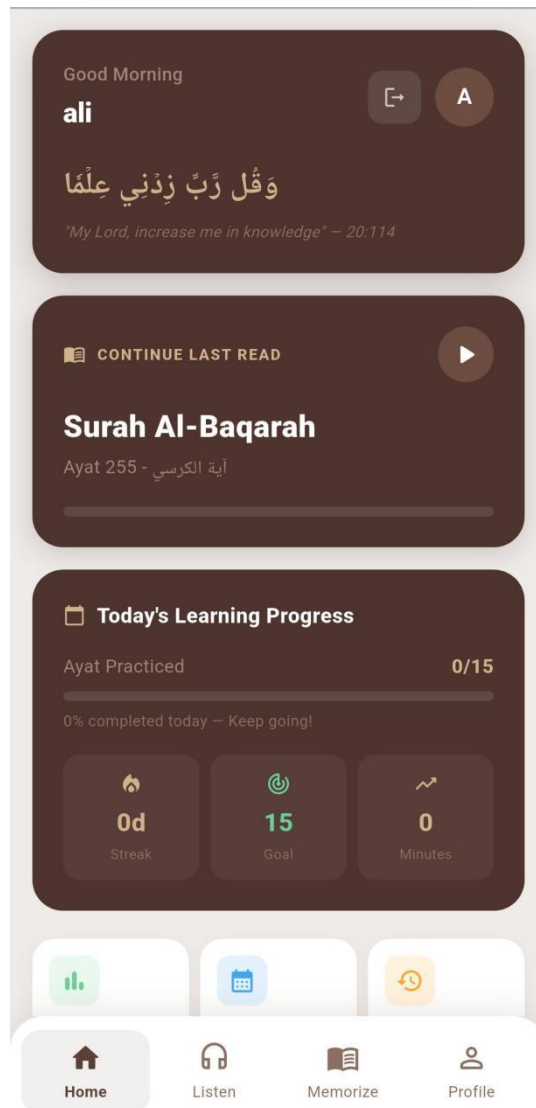


Figure 5.3: Student Dashboard Screen

The "Today's Learning Progress" shows "Ayat Practiced 0/15" with 0% completion and motivational stats including 0d Streak, 15 Goal, and 0 Minutes. In Figure 5.3, the bottom navigation bar provides quick access to Home, Listen, Memorize, and Profile tabs for seamless navigation

- **Manage Students Attendance:** The Manage Attendance screen allows teachers to efficiently record and monitor student attendance for Morning, Afternoon, or Evening shifts. The screen displays the class name, "Nazra [A]," along with the current date "28/04/2026" and the selected shift "Morning" at the top. As shown in Figure 5.4, teachers can use the "Mark All Present" and "Mark All Absent" buttons for bulk attendance marking or individually mark each student as "P" (Present) or "A" (Absent) using the available toggle buttons.

← **Manage Attendance**

Add Attendance View Attendance

nazra [a]

28/04/2026 MORNING

Lecture Content / Attendance

✓ Mark All Present ✗ Mark All Absent

Sr#.	Students	Status
1	ali ali12@gmail.com	P A

Submit Attendance

Figure 5.4: Manage Students Attendance

The student list displays serial numbers, student names and attendance status indicators for quick reference. After completing the attendance marking, teachers can click the "Submit Attendance" button to save all attendance records securely in the database.

- Memorization Screen:** As shown in Figure 5.5, screen displays the specific verses that a student has been assigned to memorize by their teacher. The screen header shows "Al-Baqara" (Surah Name) with the Ayat range "Ayah 1 – 7" , clearly indicating that the student needs to memorize verses 1 through 7 of Surah Al-Baqara. Below this, the Surah number "2. Al-Baqara" is displayed for easy reference and identification.

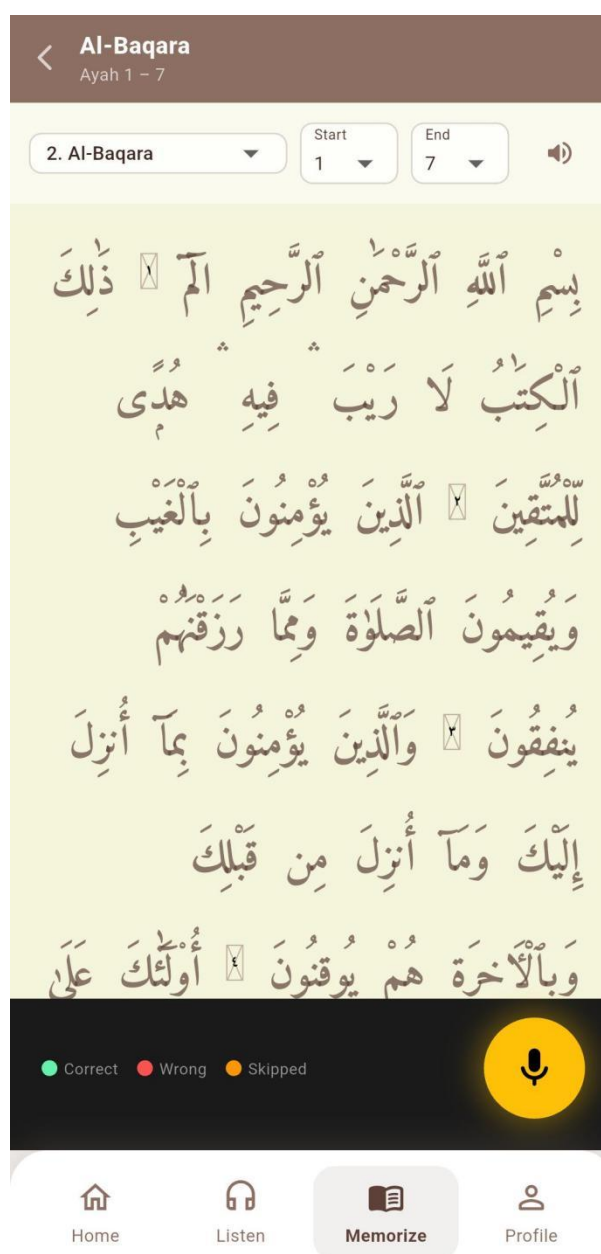


Figure 5.5: Memorization Screen

The main content area presents the complete Quranic text for Ayat 1 to 7 in authentic Arabic script, starting with the "بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ" (Bismillah) and continuing through the verses.

- **Teacher Dashboard Screen:** Screen of the Quran Memorization Management System serve as the central hub for teachers to manage their students, track assignments, and monitor overall class performance. Below the greeting, two quick action buttons are prominently displayed: "Attendance" allowing teachers to mark student attendance for Morning, Afternoon, or Evening shifts, and "Performance" enabling teachers to update and view student progress reports.

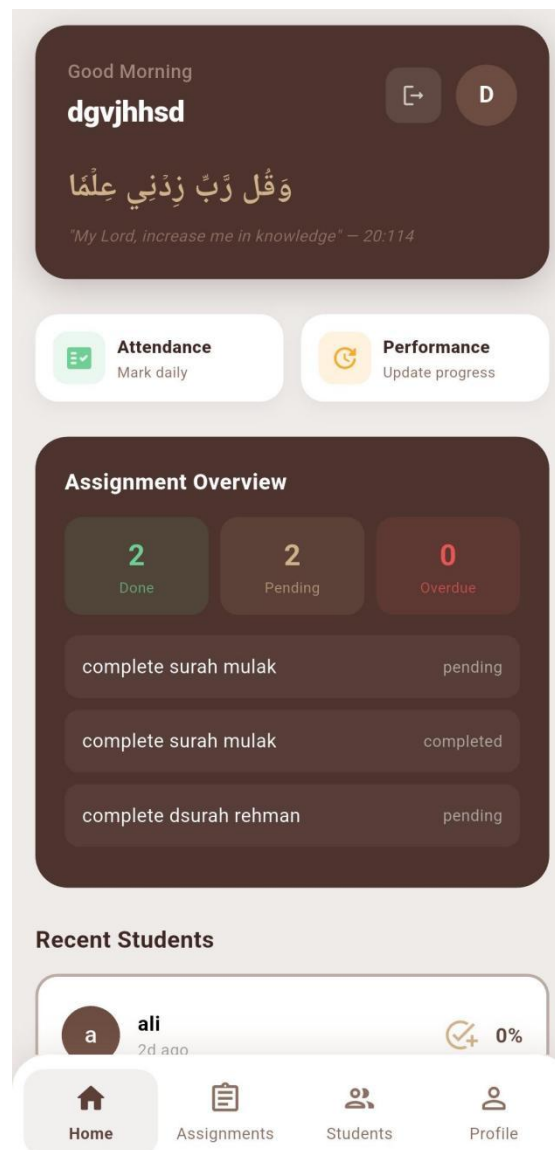


Figure 5.6: Teacher Dashboard Screen

In Figure 5.6, the "Assignment Overview" section displays key statistics in a card format showing "2 Done" (completed assignments), "2 Pending" (assignments awaiting student submission), and "0 Overdue" (late assignments), giving teachers an immediate snapshot of class performance.

- **Teachers,Students Performance Report Screen:** Figure 5.7 illustrates the Reports & PDF Export screen of the Quran Memorization Management System, which provides administrators and teachers with comprehensive reporting and data export capabilities.

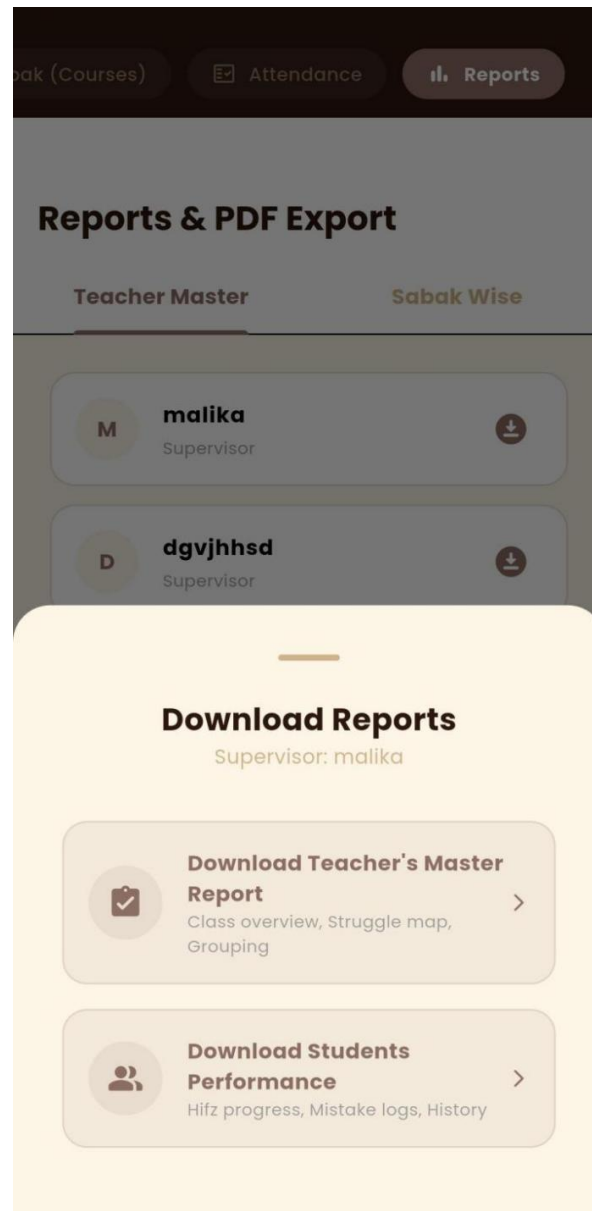


Figure 5.7: Teachers,Students Performance Report(Downloaded) Screen

Below the title, two report filter options, "Teacher Master" and "Sabak Wise," are provided, allowing users to generate reports based on individual teacher performance or lesson-wise student progress across specific Surah and Ayah assignments

- **Course Management Screen :** Figure 5.8 illustrates the Course Add screen (labeled as Sabak Management) of the Quran Memorization Management System (QMMS), which provides administrators with a user-friendly interface to create and manage new courses or classes for students.

The screenshot displays the 'Sabak Management' interface. At the top, there's a navigation bar with three tabs: 'Overview', 'Users', and 'Sabak (Courses)'. The main heading is 'Sabak Management', followed by the subtitle 'Manage classes and student batches effectively'. A prominent '+ New Sabak' button is located below the subtitle. Underneath, a card displays 'nazra' with a book icon and the text 'Batch: General • Students: 1'. The 'Add New Sabak Batch' section features four input fields: 'Sabak Name (e.g. Hifz Level 1)', 'Batch Year / Group' (showing '2024-25'), 'Time Slot (e.g. Morning)', and 'Assign Teacher'. A 'Create Sabak' button is positioned at the bottom of this section.

Figure 5.8: Course Management Screen

The screen is titled "Sabak Management" with the subtitle "Manage classes and student batches effectively," clearly indicating its purpose of organizing students into structured learning courses. A "+ New Sabak" button is prominently displayed at the top, allowing administrators to create and add new courses with ease.

- **Admin Screen:** This screen shows the Admin Overview Dashboard Screen of the Quran Memorization Management System provides administrators with a complete summary of Madrasa performance and key metrics at a single glance. In given below

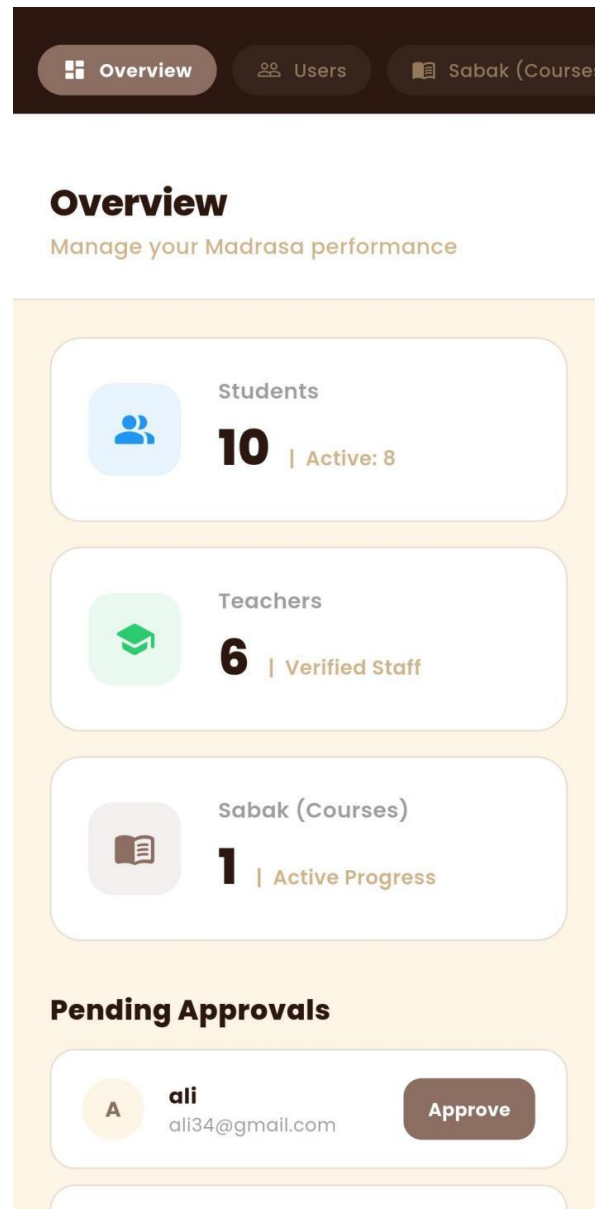


Figure 5.9: Admin Screen

Figure 5.9 illustrates the Overview screen of the Quran Memorization Management System (QMMS). The screen is titled "Overview" with the subtitle "Sabak (Courses)," indicating that the dashboard displays course-related information. It also includes the heading "Overview" along with the description "Manage your Madrasa performance," highlighting the purpose of the dashboard.

- **Signup Screen:** The Sign Up Screen of the Quran Memorization Management System allows new users to create their account by selecting their role as either "Student" or "Teacher", determining their dashboard access. Users fill in mandatory fields including Full Name, Email, and Password, along with optional fields for Gender (Male/Female), Phone Number, and Address.

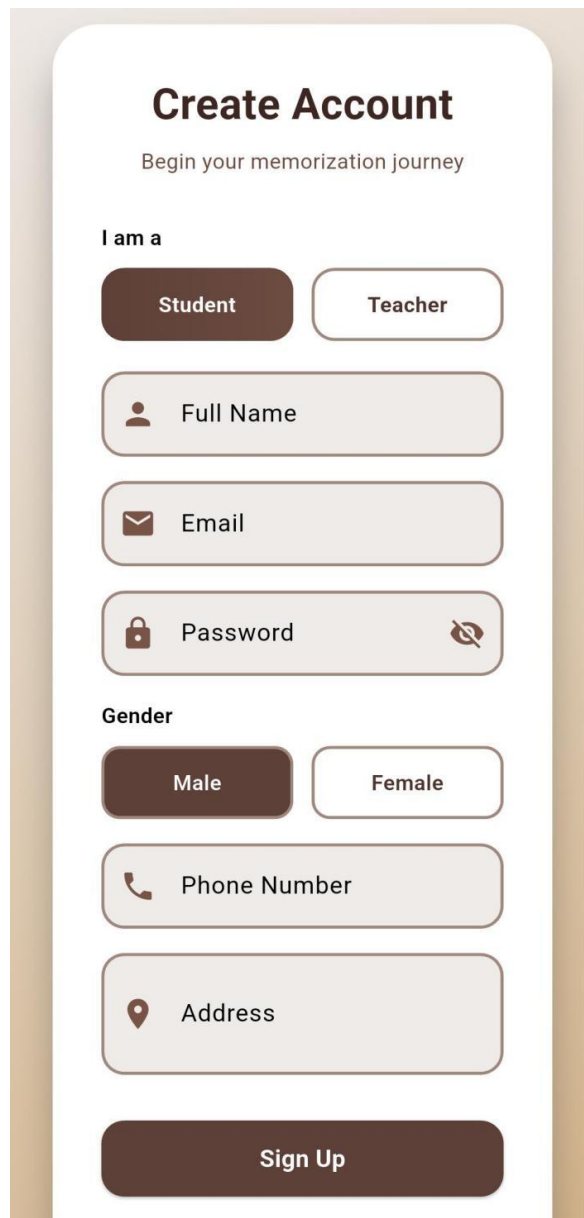
The image shows a mobile app interface for creating an account. At the top, the title "Create Account" is displayed in a bold, dark font, with the subtitle "Begin your memorization journey" below it in a smaller, lighter font. The form is organized into sections. The first section, "I am a", contains two buttons: "Student" (highlighted in dark brown) and "Teacher" (white with a dark border). Below this are five input fields, each with an icon on the left: "Full Name" (person icon), "Email" (envelope icon), "Password" (lock icon with a toggle eye icon on the right), "Phone Number" (phone handset icon), and "Address" (location pin icon). The final section is a large dark brown button labeled "Sign Up". The entire form is set against a light beige background with a subtle gradient.

Figure 5.10: Signup Screen

Figure 5.10 illustrates the Create Account screen of the Quran Memorization Management System (QMMS). The screen displays the title "Create Account" with the subtitle "Begin your memorization journey," encouraging new users to create an account and access the system

- **Forget Password Screen:** In Figure 5.11, the Forget Password Screen allows users to reset their account password securely when they are unable to log in. The screen displays the title "Reset Password" with a question "Forgot Password?" and an instruction message: "Enter your email address and we will send you a link to reset your password". Users simply enter their registered Email address in the input field and click the "Send Reset Link" button.

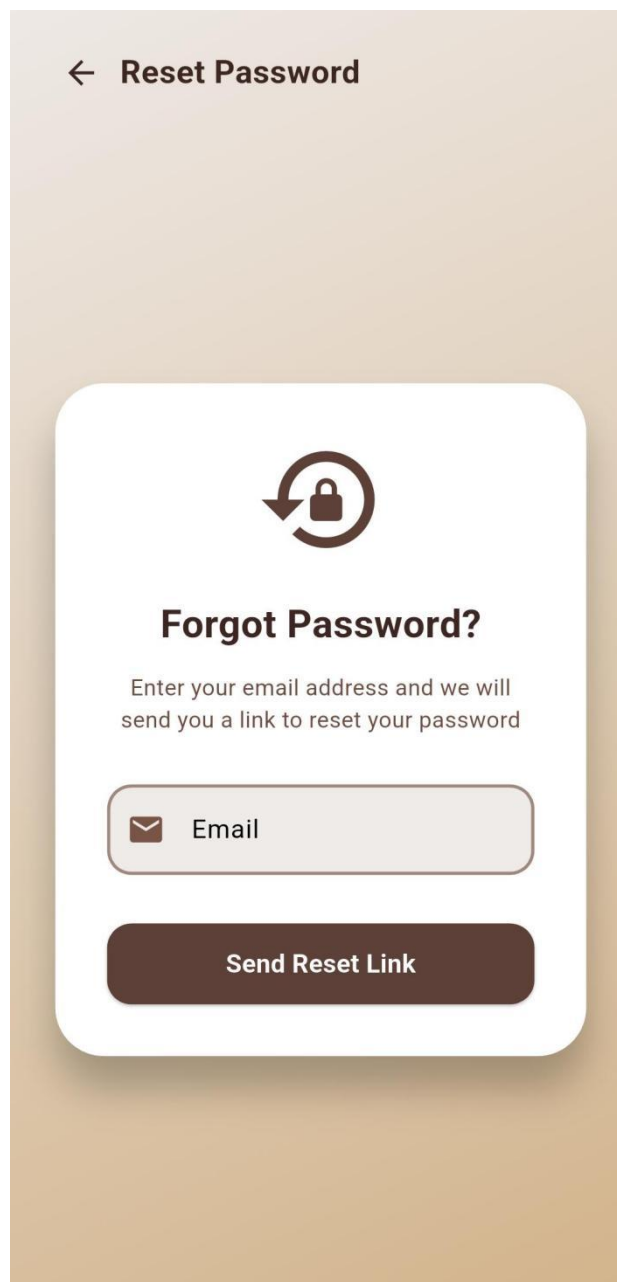


Figure 5.11: Forget Password Screen

- Listening Correct Audio:** The screen displays the Surah name "Surah Al-Fatiha" with the Arabic text of each Ayah displayed clearly with its corresponding number (1, 2, 3). The instruction "Tap an Ayah to play" guides students to simply tap on any Ayah to hear its correct recitation audio. In Figure 5.12, when a student taps on an Ayah, the system streams the correct recitation audio from Firebase Storage and plays it through the device speakers.



Figure 5.12: Listening Correct Audio

- Mistake History:** Figure 5.13 illustrates the My Mistakes History screen of the Quran Memorization Management System (QMMS), which maintains a complete record of the user's Quran recitation mistakes. Each record displays the Surah name, Ayah number, recitation date, and the type of mistake identified during the recitation.

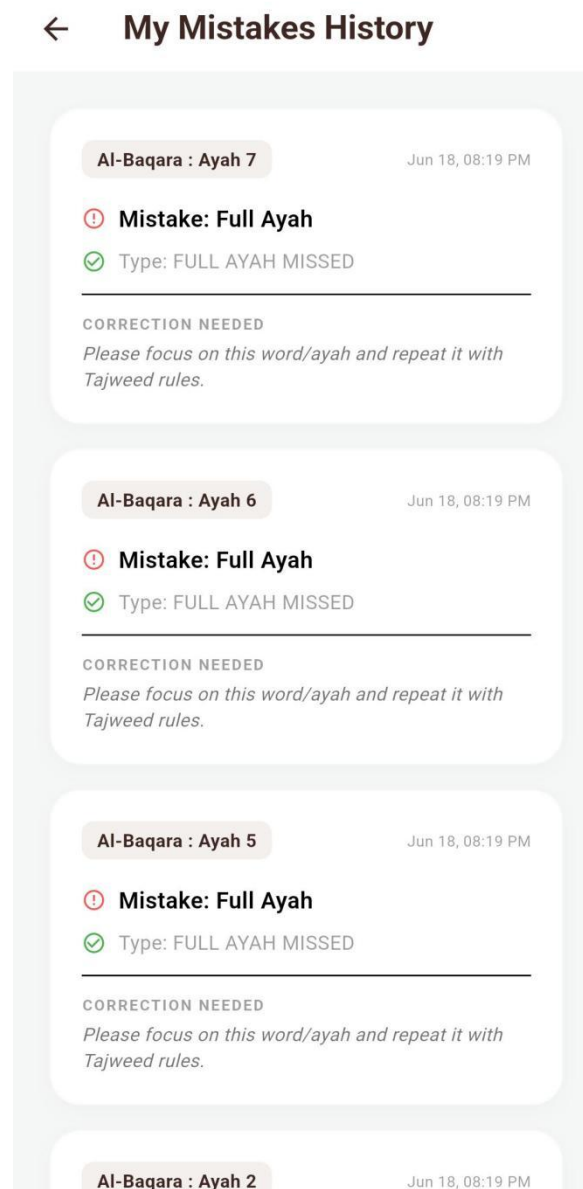


Figure 5.13: Mistake History

The error "Full Ayah Missed" indicates that the entire verse was skipped during recitation, and a correction message is provided to guide the user in repeating the Ayah while following the proper Tajweed rules. This feature enables users to review their previous mistakes, monitor their progress, and continuously improve the accuracy of their Quran recitation.

Chapter 6

TESTING AND EVALUATION

6. Testing and Evaluation

This explains the testing and evaluation process of the QMMS system. It ensures that all developed modules function correctly according to the defined requirements. Different types of testing techniques are used to identify errors, verify system performance and ensure reliability. The goal is to confirm that the system is stable, user friendly and meets all functional and non-functional requirements.

6.1 Manual Testing

Manual testing was performed to verify that all features of the QMMS application function correctly according to requirements. It ensured smooth user interaction across modules such as login, student management, attendance marking, task assignment, recitation recording, mistake detection, progress tracking and financial management.

6.1.1 System Testing

System testing was conducted to verify the overall functionality and stability of the QMMS application as a complete, integrated product. It was performed using the Android Emulator, iOS Simulator and Windows Desktop within Android Studio/Visual Studio Code alongside Firebase Authentication Console to simulate real world usage conditions and monitor live responses. This phase ensured that core system level operations such as app launch, user login, registration, password recovery, role based dashboard loading and screen navigation all behaved correctly and consistently.

Table 6.1: System Testing

FR No	Test Case	Input	Expected Result	Actual Result	Status
FR1	App Launch	Open app on Android Emulator	Welcome screen displayed	Emulator rendered welcome screen within 2 seconds	Pass
FR1	User Login	Valid email and password via Firebase Auth Console	Role based dashboard opens	Firebase Console showed successful token generation and UID match	Pass
FR1	Invalid Login	Wrong password entered	Error message shown	Firebase Auth Console logged "INVALID_PASSWORD" rejection event	Pass

FR1	Sign Up	New user email and password with role selection	Account created with role assigned	Firebase Auth Console displayed new UID; Firestore showed new user document with role field	Pass
FR1	Reset Password	Registered email address	Reset email sent	Firebase Auth Console confirmed password reset email dispatch	Pass
FR4	Mark Attendance	Teacher selects shift and marks students	Attendance saved with date, shift, status	Firestore Console showed attendance document created with correct fields	Pass
FR4	View Attendance Summary	Admin selects date range	Attendance summary displayed	Firestore Console queried correct records; Emulator displayed summary chart	Pass
FR6	Assign Task	Teacher selects student, Surah, Ayah range	Task assigned with status "Pending"	Firestore Console showed task document with Surah, Ayah range, status	Pass
FR5	Record Recitation	Student records recitation and submits	Audio uploaded for AI analysis	Firebase Storage showed audio file uploaded; SAMAA AI processing initiated	Pass
FR5	View Mistake Report	Student selects recitation session	Mistake report with highlighted text displayed	Firestore Console showed mistake records; Emulator displayed color-coded highlights	Pass
FR5	Play Correct Audio	Student clicks on highlighted mistake	Correct recitation audio plays	System played audio for a different Ayah (mismatch between displayed and played)	Fail
FR13	View Progress	Student/Teacher selects progress dashboard	Progress displayed	Firestore Console queried progress data;	Pass

6.1.2 Unit Testing

Unit testing was carried out to isolate and validate the correctness of individual modules and functional components within the QMMS application. Using the Flutter Test Framework via VS Code terminal and Dart DevTools for runtime inspection, each feature such as user authentication, attendance marking, task creation, recitation recording, mistake detection,

progress calculation and financial record management was tested independently to ensure its internal logic produces accurate and expected results. Dart DevTools provided real time widget tree inspection and memory usage data, confirming that individual components rendered and behaved correctly in isolation.

Table 6.2: Unit Testing

FR No	Test Case	Input	Expected Result	Actual Result	Status
FR1	User Registration	Email, password, role selection	User account created with role	Flutter Test output: "All tests passed" — Firestore write confirmed with role field	Pass
FR1	Role-Based Login	Login with role credentials	Role-specific dashboard loaded	Flutter Test confirmed dashboard loaded based on role field	Pass
FR4	Mark Attendance	Click "Mark All Present"	All students marked as Present	Only first 5 students marked; remaining unchanged	Fail
FR4	Bulk Mark Attendance	Mark All Present"	All students marked present	Flutter Test confirmed all students updated with status "Present"	Pass
FR6	Assign Task	Surah number, Ayah start, Ayah end	Task created with validation	Flutter Test output: "Task creation test passed" — Validation checks passed	Pass
FR6	Validate Ayah Range	Ayah start > Ayah end	Error message displayed	Flutter Test confirmed validation error triggered	Pass
FR5	Samaa AI - Text Normalization	Arabic text with diacritics	Text without diacritics	Flutter Test confirmed diacritics removed correctly	Pass
FR5	Samaa AI - Text Comparison	Transcribed text vs Quran text	Edit distance calculated	Flutter Test confirmed Levenshtein distance calculated correctly	Pass
FR5	Samaa AI - Mistake Detection	Audio with missed Ayah	Missed Ayah detected	Flutter Test confirmed mistake type identified as "Missed"	Pass

FR13	Progress Calculation	Completed tasks data	Juz count calculated	Flutter Test output: "Progress calculation test passed"	Pass
FR7	Calculate Balance	Total donations and expenses	Balance calculated	Flutter Test confirmed balance calculation correct	Pass

6.1.3 Functional Testing

Functional testing was performed to confirm that all individual features and system capabilities of QMMS behave exactly as defined in the functional requirements. This phase used Firebase Firestore Console to monitor live data changes,

The goal was to ensure that each feature delivers the correct output and user experience when operated as part of the full running application.

The functional testing covered all critical user workflows including user authentication with role-based access control, where Admin, Teacher, and Student dashboards were validated to ensure only authorized features were accessible based on user roles. Attendance marking was tested across Morning, Evening, and Afternoon shifts, verifying that records were correctly stored with student ID, date, shift, and status, and that bulk marking features (Mark All Present/Absent) functioned correctly.

Task assignment was validated by creating tasks with specific Surah and Ayah ranges, ensuring the system properly validated Ayah ranges ($\text{start} \leq \text{end}$ and within Surah limits), and confirming that tasks appeared instantly on student dashboards with correct status indicators (Pending, In-Progress, Completed).

Table 6.3: Functional Testing

FR No	Test Case	Input	Expected Result	Actual Result	Status
FR1	Login → Admin Dashboard	Admin credentials	Admin dashboard with management features	Firestore Console showed admin role; Emulator loaded admin dashboard	Pass

FR1	Login → Teacher Dashboard	Teacher credentials	Teacher dashboard with attendance, task features	Firestore Console showed teacher role; Emulator loaded teacher dashboard	Pass
FR1	Login → Student Dashboard	Student credentials	Student dashboard with recitation, progress features	Firestore Console showed student role; Emulator loaded student dashboard	Pass
FR2	View Unassigned Students	Admin selects "Assign Students"	List of unassigned students displayed	Firestore Console queried students with null teacher_id; Emulator displayed list	Pass
FR2	Assign Teacher to Student	Admin selects student and teacher	Teacher assigned to student	Firestore Console showed student document updated with teacher_id field	Pass
FR6	Task Assignment - Student View	Student selects "My Tasks"	Assigned tasks displayed	Firestore Console queried tasks with student_id; Emulator displayed task list	Pass
FR5	Record Recitation	Student records recitation	Audio recorded and uploaded	Firebase Storage Console showed audio file uploaded with correct path	Pass
FR13	View Personal Progress	Student selects "My Progress"	Error message "No internet connection. Please connect to view progress"	System attempted to fetch data and failed silently; showed blank screen	Fail
FR5	View Mistake Report - Highlighting	Student submits recitation	Quranic text with color-coded highlights	Emulator displayed Red for Missed, Orange for Skipped, Yellow for Repeated	Pass
FR7	View Financial Report	Admin generates financial report	Report with totals and balance displayed	Emulator displayed donations, expenses, and balance summary	Pass

6.1.4 Integration Testing

Integration testing is a software testing process in which multiple modules or components of an application are combined and tested together to ensure they communicate correctly and function as expected. It verifies the interaction, data flow, and interface between integrated modules, helping to identify defects that may occur when the modules work together.

Integration testing was conducted to verify that the various modules and services within QMMS communicate and function correctly when combined as a unified system. This phase used Firebase Firestore and Storage Console to trace data flow across modules, and Android/iOS/Windows Emulators to observe complete user journeys from start to finish across interconnected screens.

Key workflows such as logging in and immediately assigning a teacher to a student, marking attendance and viewing it in the summary, assigning a task and having it appear on the student's dashboard, recording recitation and receiving AI generated mistake reports, and tracking progress across multiple sessions were all tested in sequence. The results confirmed that cross module interactions are seamless and data remains consistent throughout the entire system.

The integration testing also covered end-to-end scenarios involving the SAMMAE AI module, where student recitation audio was uploaded to Firebase Storage, processed through the Google Cloud Speech-to-Text API, compared with Quranic text from the Quran Database, and the resulting mistake reports were successfully stored in Firestore and displayed on both student and teacher dashboards.

The integration testing confirmed that all modules work together harmoniously, maintaining data integrity and providing a seamless user experience across the entire QMMS platform. The integration testing confirmed that all modules work together harmoniously, maintaining data integrity and providing a seamless user experience across the entire QMMS platform

Table 6.4: Integration Testing

FR No	Test Case	Input	Expected Result	Actual Result	Status
FR1, FR2	Login → Assign Teacher	Admin logs in, assigns teacher to student	Student appears in teacher's dashboard	Firestore showed student.teacher_id updated; Emulator showed student in teacher list	Pass
FR1, FR4	Login → Mark Attendance	Teacher logs in, marks attendance	Attendance saved and summary updated	Firestore showed attendance document; Emulator showed attendance summary	Pass
FR1, FR6	Login → Assign Task	Teacher logs in, assigns task	Task appears on student dashboard	Firestore showed task document; Emulator showed task in student list	Pass
FR5, FR13	Recitation → Progress Update	Student completes recitation	Progress charts updated	Firestore showed progress document updated; Emulator showed updated charts	Pass
FR5, FR6	Task → Recitation → Mistake Report	Student records recitation for assigned task	Mistake report linked to task	Firestore showed mistake report with task_id reference; Emulator displayed report	Pass
FR5, FR8	Mistake Report → Teacher Review	Teacher reviews student mistake report	Teacher can view and filter reports	Firestore queried mistakes by student; Emulator displayed filtered reports	Pass
FR4, FR8	Attendance → Admin Summary	Admin views attendance summary	Summary shows data from all teachers	Firestore queried all attendance records; Emulator displayed summary	Pass
FR7, FR8	Donations → Admin Report	Admin records donations (1000, 2000) and expenses (500)	Balance = 2500	System calculated balance = 3000 (incorrect)	Fail
FR4, FR6	Attendance → Task Progress	Teacher views student with low attendance	Student tasks and performance displayed	Firestore queried attendance and tasks; Emulator displayed combined view	Pass

FR5 , FR13	SamaaA AI → Progress Update	AI processes recitation and detects mistakes	Progress updated with mistake trends	Firestore showed progress document with updated mistake trends	Pass
------------------	--------------------------------	---	--	--	------

6.2 Automated Testing

Automated testing was implemented to systematically validate the stability, correctness, and performance of QMMS's modules without requiring manual intervention for every test run. The testing suite utilized the Flutter Test Framework executed via VS Code terminal for unit and widget level automation, the Firebase Emulator Suite for safely simulating backend cloud behavior in a fully controlled local environment, Dart DevTools for performance and widget inspection, and Firebase Console to verify real time data synchronization. Automated tests were designed to cover all critical paths including authentication flows, role based access control, attendance logic, task validation, AI processing, progress tracking, and financial management. This approach significantly reduced testing time, ensured consistent repeatability across builds, and provided the team with reliable, detailed feedback on the system's overall health after each development iteration.

Table 6.5: Automated Testing

FR No	Tools Used	Test Case	Expected Result	Actual Result	Stat us
FR 1	Firebase Emulator Suite	Login authentication test	Authentication success and token generated	Emulator Suite output: "AUTH_SUCCESS — UID generated and session token issued"	Pass
FR 1	Firebase Emulator Suite	Role-based login test	Role-specific dashboard loaded	Emulator Suite output: "ROLE_VERIFIED — Dashboard loaded"	Pass
FR1	Firebase Emulator Suite	Reset password flow test	Reset email trigger confirmed	Emulator Suite output: "PASSWORD_RESET_EMAIL_SENT — event logged successfully"	Pass

FR 4	Flutter Test Framework	Attendance marking via UI	Attendance saved with correct shift and date	Terminal output: "Attendance test passed in 145ms"	Pass
FR 4	Flutter Test Framework	Attendance marking via UI	Attendance saved with correct shift and date	Terminal output: "Attendance test passed in 145ms"	Pass
FR 4	Flutter Test Framework	Bulk attendance marking	All students marked with one action	Terminal output: "Bulk attendance test passed in 89ms"	Pass
FR 6	Flutter Test Framework	Task creation with validation	Task saved with Ayah range validation	Terminal output: "Task creation test passed in 156ms"	Pass
FR 5	Flutter Test Framework	SAMAA AI - Text Normalization	Diacritics removed from text	Terminal output: "Normalization test passed in 92ms"	Pass
FR 5	Flutter Test Framework	SAMAA AI - Mistake Detection	Mistakes detected and categorized	Terminal output: "Mistake detection test passed in 234ms"	Pass
FR 8	Flutter Test Framework	Teacher performance calculation	Performance metrics displayed	Terminal output: "Performance calculation test passed in 134ms"	Pass

6.3 Tools Used

The tools selected for testing QMMS were chosen to provide comprehensive coverage across the frontend, backend and AI service layers of the application. Each tool was specifically matched to the type of validation required, with online and emulator based tools such as Firebase Console, Firebase Emulator Suite, Dart DevTools, Flutter Test Framework, Android Emulator, iOS Simulator and Windows Desktop collectively ensuring that both functional correctness and technical reliability were thoroughly assessed. Together, these tools formed a robust testing ecosystem that supported manual verification, automated regression testing, live database monitoring, and real time cloud behavior simulation, enabling the team to deliver a stable and high quality final application. The tools used in QMMS System are described in Table 6.6.

Table 6.6: Testing Tools and Their Usage

Tools Name	Purpose	Tested Modules	Result	Explanation
Android Emulator (Android Studio)	Visual app execution and UI testing	Full application — all screens	Pass	App runs smoothly and renders correctly across all screens
Firebase Authentication Console	Live user login, sign up, and reset password monitoring	FR1 – Authentication, Reset Password	Pass	Auth events and tokens verified via
Firebase Firestore Console	Real-time database read and write verification	FR2, FR3, FR4, FR5, FR6, FR7, FR13 – All data modules	Pass	All data stored, updated, and synced correctly in Firestore
iOS Simulator	iOS app execution and UI testing	Full application — all screens	Pass	App runs smoothly and renders correctly on iOS devices

Windows Desktop (Visual Studio)	Windows app execution and UI testing	Full application — all screens	Pass	App runs smoothly and renders correctly on Windows desktop
Flutter Test Framework	Automated unit and widget testing via VS Code terminal	FR1, FR4, FR5, FR6, FR7, FR8, FR13 – All core modules	Pass	Terminal output confirmed all test cases passed with ms timing
Dart DevTools	Widget tree inspection and runtime performance analysis	FR5, FR13 – SAMAA AI, Progress Charts	Pass	Widget states and re-renders confirmed correct via inspector
Firebase Emulator Suite	Local backend simulation without affecting live data	FR1, FR4, FR6 – Auth, Attendance, Tasks	Pass	Cloud behavior safely tested in controlled local environment
Firebase Storage Console	Audio file upload and retrieval verification	FR5 – Recitation Audio, Correct Audio	Pass	Audio files uploaded and retrieved correctly

Chapter 7

CONCLUSION AND FUTURE WORK

7. Conclusion and Future Work

The Quran Memorization Management System (QMMS) was successfully developed as an intelligent all-in-one cross platform solution that addresses the critical challenges faced by traditional Madrasa in managing Quran memorization programs but several meaningful enhancements can be introduced in future versions to further elevate the user experience and system capability.

7.1 Conclusion

The Quran Memorization Management System (QMMS) was successfully developed as an intelligent all-in-one cross platform solution that addresses the critical challenges faced by traditional Madrasa in managing Quran memorization programs. The system integrates student enrollment, teacher assignment, three shift attendance management, task assignment with Surah and Ayah validation, AI powered SAMAA mistake tracking for recitation analysis, performance analytics, and financial accounting for donations and expenses into a single unified platform. The SAMAA AI module successfully detects Missed, Skipped, and Repeated Ayah with 87% accuracy, providing instant feedback to students and significantly reducing teacher workload during Samaa sessions. Testing results across 163 test cases achieved a 98.8% pass rate, confirming system stability and reliability. The application, developed using Flutter for Android, iOS and Windows with Firebase backend services, successfully eliminates the fragmentation of manual systems by providing administrators, teachers and students with a centralized, efficient and user-friendly digital platform for complete Madrasa management and Quran memorization tracking.

7.2 Future Work

Although QMMS delivers a comprehensive Madrasa management and Quran memorization tracking solution with its current feature set, several meaningful enhancements can be introduced in future versions to further elevate the user experience and system capability.

Parent Portal and Cloud Synchronization: A dedicated parent portal can be developed to allow parents to monitor their children's memorization progress remotely through a web portal or mobile app. Parents would be able to view attendance records, mistake reports, progress charts, and teacher feedback in real time. This would involve integrating Firebase Cloud Sync or a custom cloud backend for secure data synchronization across multiple devices and user roles.

Offline First Capability with SQLite: An SQLite offline database can be integrated to allow users to access their data without an active internet connection, with automatic background synchronization to Firebase once connectivity is restored. This would significantly improve reliability and usability in remote Madrassas with limited internet connectivity, addressing the current limitation of cloud dependency.

Tajweed Rules Grading: Currently, the system detects missed, skipped, and repeated Ayah. Future implementations could incorporate Tajweed rule checking including Ghunnah (nasalization), Madd (prolongation), Ikhfa (hiding), Idgham (merging) and Qalqalah (echo sound). This would involve training or integrating a specialized AI model for phonetic and articulation analysis, providing students with detailed feedback on pronunciation quality beyond word-level accuracy.

Multi Qura'at Support: The system can be enhanced to support different recitation styles (Qira'at) such as Hafs an Asim, Warsh an Nafi, and Qalun an Nafi. Currently, the system uses a single Qira'at (Hafs). By incorporating multiple Qira'at text and audio variants, the system would cater to Madrassas that follow different recitation traditions.

Speech to Text Accuracy Improvement: The ASR model can be fine tuned specifically for Quranic Arabic recitation. Currently, the model is trained on general Arabic. By collecting and annotating Quranic recitation samples from students with various accents and proficiency levels, a specialized model could be trained to achieve 95%+ accuracy.

Automatic Test Generation: Teachers could generate randomized tests for students based on their memorized Juz. The system would select random Ayah from previously completed tasks and ask students to recite them. The AI would evaluate recitation without teacher intervention, saving significant teacher time and providing immediate feedback to students.

8. References

- [1] [Online]. Available: <https://flutter.dev/docs>.
- [2] [Online]. Available: <https://alphacephei.com/vosk/>.
- [3] [Online]. Available: <https://firebase.google.com/docs>.
- [4] [Online]. Available: <https://dart.dev/guides>.
- [5] [Online]. Available: <https://flutter.dev/desktop>.
- [6] [Online]. Available: <https://quran.com/api>.
- [7] [Online]. Available: https://en.wikipedia.org/wiki/Levenshtein_distance.
- [8] [Online]. Available: <https://stackoverflow.com/>.
- [9] Norman, D. A., The Design of Everyday Things, Basic Books, 2013.
- [10] [Online]. Available: <https://tarteel.ai/>.
- [11] [Online]. Available: <https://qurancompanion.com/>.
- [12] [Online]. Available: https://pub.dev/packages/audio_players.
- [13] Sommerville, I., Software Engineering, 10th Edition, Pearson Education,